Applied Reinforcement Learning

Value-based methods

Riccardo Berta

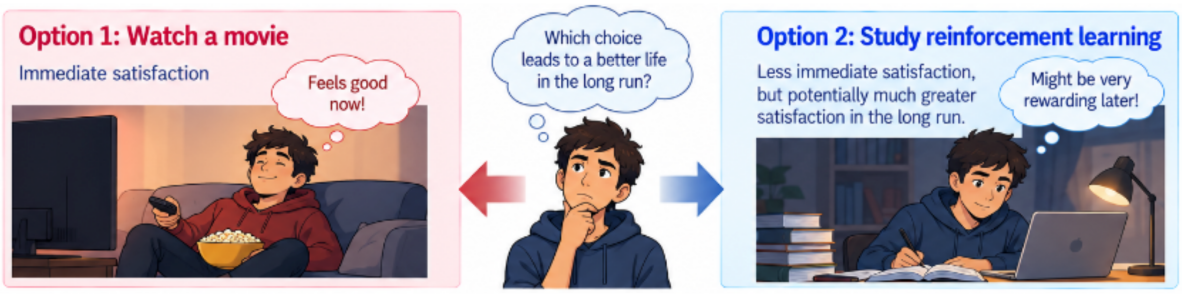
2026.10.05

Contents

1 Value-based Methods **1**
1.1 Policy Evaluation . . . 2
1.1.1 Random Walk Environment . . . 3
1.1.2 Improving Estimates after Each Episode: Monte Carlo . . . 4
1.1.3 First-visit and Every-visit Monte Carlo . . . 9
1.1.4 Improving estimates after each step: Temporal-Difference Learning . . . 14
1.1.5 Improving estimates after multiple steps: n-steps TD . . . 20
1.1.6 Improving estimates of all visited states: TD(λ) . . . 26
1.1.7 The Backward View: Eligibility Traces . . . 28
1.2 Policy Improvement . . . 34
1.2.1 Slippery Walk Environment . . . 35
1.2.2 Improving policies after each episode: Monte Carlo control . . . 38
1.2.3 Improving policies after each step: SARSA . . . 44
1.2.4 Convergence requirements for on-policy control . . . 49
1.2.5 Decoupling behavior from learning: Q-Learning . . . 50
1.2.6 Convergence requirements for off-policy control . . . 56
1.2.7 Solving the maximization bias: Double Q-Learning . . . 57
1.2.8 Using multi-step estimates: SARSA(lambda) . . . 63
1.2.9 Using multi-step estimates and decoupling behaviour from learning: Q(lambda) . . . 69
1.3 Comparison . . . 74

1 Value-based Methods

Balancing **immediate** and **long-term goals** is challenging. We experience this tension every day: should I watch movies tonight, or keep studying reinforcement learning? The first choice gives us **immediate satisfaction**. The second may not give us much satisfaction tonight, but maybe (and only maybe) it will provide much **greater satisfaction in the long run**. But how much greater? We do not know, and we cannot know without trying it, without exploring. Life does not give us its MDP! We do not know exactly how our actions will affect the future, and even the same action may lead to different outcomes. This brings together the two fundamental challenges we have studied so far. We must **balance information gathering and information utilization**, while at the same time **reasoning about the long-term consequences** of our actions rather than only their immediate effects. In other words, we must deal simultaneously with exploration versus exploitation and immediate versus long-term reward.



We will now study agents that can learn to estimate the value of policies, much like the policy-evaluation methods we encountered in dynamic programming, but with one crucial difference: this time, the agent does not know the MDP. **It must learn values from experience**. Before doing so, let us revisit three closely related but fundamentally different concepts:

- The **reward** is the one-step feedback signal received from the environment. The agent observes a state, selects an action, and receives a reward. Rewards tell us what is good immediately, but maximizing the reward received at each individual step is not the objective of reinforcement learning. A greedy pursuit of immediate reward can sacrifice much larger rewards available later.
- The **return** is the accumulated discounted reward from a given time onward. Unlike a single reward, it captures the long-term consequences of a sequence of decisions. However, in a stochastic environment, the return itself is a random variable. The same policy, starting from the same state, may produce a very high return in one episode and a very low return in another. Therefore, optimizing for one particular realized return is not enough.
- The **value function** captures what we really need: the expected return. It tells us how much discounted reward we can expect to accumulate in the future, on average, when following a particular policy. Random transitions and stochastic policies are therefore naturally accounted for: individual trajectories may differ, but their expected return defines the value.

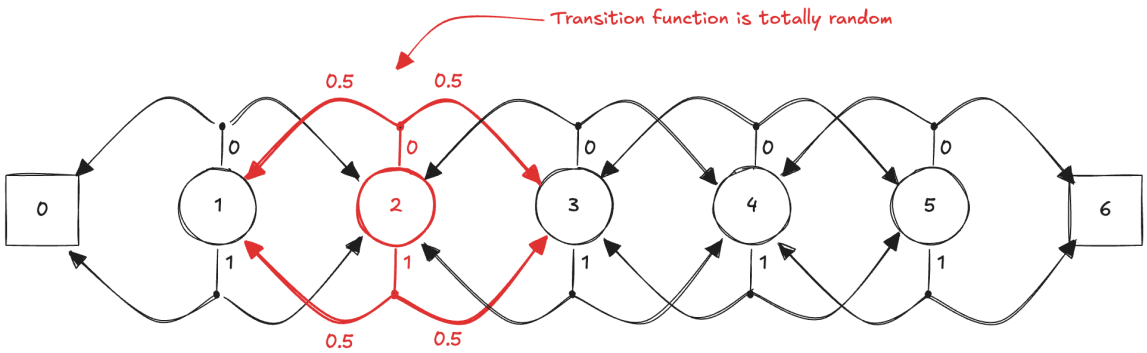
The central idea is therefore simple: the agent is not trying to maximize the immediate reward, nor a single realized return. It is trying to find behavior that maximizes the **expected return** and **value functions allow us to reason about exactly that quantity**.

1.1 Policy Evaluation

We begin with the simplest question: suppose the policy is fixed, how good is it? Answering this question means estimating its value function from sampled experience, without knowing the environment model. This is the **prediction problem**: learning the expected return obtained by following a given policy.

1.1.1 Random Walk Environment

Consider a single-row grid-world with five non-terminal states and two terminal states. At each step, the agent can select either the left or right action. However, the environment is constructed so that both actions have exactly the same effect: regardless of the action selected, the agent moves left with probability 0.5 and right with probability 0.5.



Therefore, the agent has **no actual control** over where it goes. Different actions do not induce different transition probabilities, so changing the policy cannot change the agent's behavior or improve its performance.This apparently artificial property is precisely what makes the environment useful for studying policy evaluation. Since there is no meaningful control problem, we can **focus entirely on the prediction problem**: learning the value of each state from sampled experience. In particular, there is no need to disentangle the stochasticity introduced by the policy from that introduced by the environment dynamics. All uncertainty about the next state comes from the environment, allowing us to concentrate on the central question: **How can an agent learn the expected return from each state using only experience?**

```
import numpy as np

# Seven-state random-walk environment.
class RandomWalk:

    # State index:  [ 0    1    2    3    4    5    6 ]
    # State label:  [ T    A    B    C    D    E    T ]
    # Initial state:[ .    .    .    C    .    .    . ]

    observation_space = 7

    # True values for the equiprobable random walk
    v_true = np.array([0.0, 1/6, 2/6, 3/6, 4/6, 5/6, 0.0])
```

```
    def __init__(self):
        self.state = None
        self.terminated = False
        self.reset()

    def reset(self):
        self.state = np.random.choice([1, 2, 3, 4, 5])
        self.terminated = False
        return self.state

    def step(self, action):
        # The action is intentionally ignored:
        # left and right transitions are equiprobable.
        self.state += np.random.choice([-1, 1])

        reward = 0

        if self.state == 0:
            self.terminated = True

        elif self.state == 6:
            self.terminated = True
            reward = 1

        return self.state, reward, self.terminated, False, {}

random_walk = RandomWalk()
```

Because the environment dynamics make the policy being evaluated irrelevant, we can choose any policy. For simplicity, we use an all-left policy:

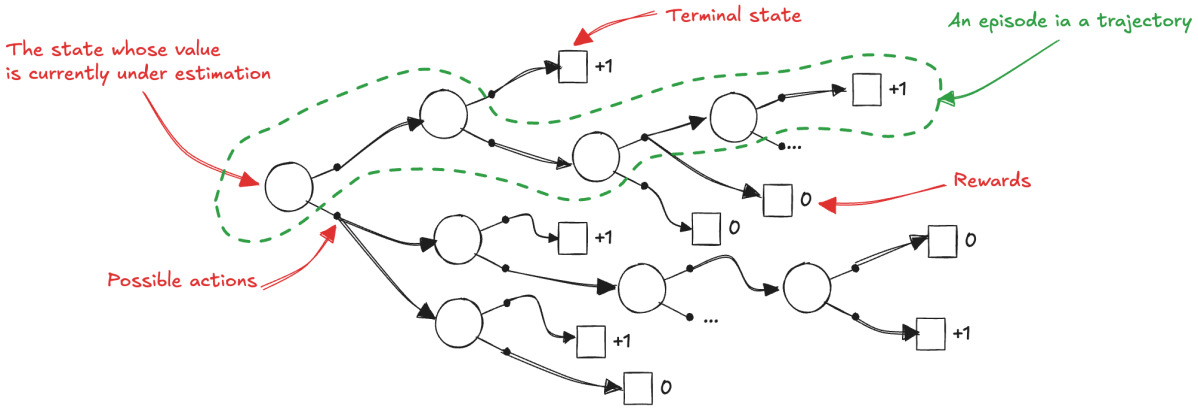
```
def pi(state):
    return 1
```

1.1.2 Improving Estimates after Each Episode: Monte Carlo

Our goal is to estimate the state-value function of a fixed policy, that is, the expected return obtained when starting from state s and then following the policy:

$v_\pi(s) = \mathbb{E}_\pi[G_{t:T} \mid S_t = s]$

A natural way to estimate an expectation is through **sampling and averaging**. We run the policy for many episodes, collect trajectories, and average the returns observed after visiting each state.



We first need a function that generates one complete trajectory by interacting with the environment according to the policy:

```python
def generate_trajectory(pi, env, max_steps=200):

    # Store all experiences collected during the episode
    trajectory = []

    # Start a new episode and obtain the initial state
    state = env.reset()

    # Interact with the environment until termination
    # or until the maximum number of steps is reached
    for _ in range(max_steps):

        # Select an action according to the policy being evaluated
        action = pi(state)

        # Apply the action and observe the environment's response
        next_state, reward, terminated, truncated, info = env.step(action)

        # The episode ends either by reaching a terminal state
        # or because the environment truncates it
        done = terminated or truncated

        # Store the complete experience generated at this time step
        trajectory.append((state, action, reward, next_state, done))
```

```
        # Stop collecting experiences when the episode ends
        if done:
            break

        # Move to the next state and continue the episode
        state = next_state

    # Return the complete trajectory as a NumPy object array
    return np.array(trajectory, dtype=object)
```

This is the basic idea behind **Monte Carlo prediction** (**MC**): estimate state values directly from **complete returns observed through experience**. Consider an episode generated by following the policy:

$S_t, A_t, R_{t+1}, S_{t+1}$

$S_{t+1}, A_{t+1}, R_{t+2}, S_{t+2}$

...

$S_{T-1}, A_{T-1}, R_T, S_T$

Each state visit provides a different return, obtained by accumulating all subsequent discounted rewards until termination:

$G_{t:T} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T$

$G_{t+1:T} = R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots + \gamma^{T-t-2} R_T$

$\vdots$

$G_{T-1:T} = R_T$

Thus, a single episode provides **one return sample for each state visit**:

$S_t \to G_{t:T}, \qquad S_{t+1} \to G_{t+1:T}, \qquad \ldots$

Computing each return independently would be wasteful. Consecutive returns satisfy the recursion:

$$\boxed{G_{t:T} = R_{t+1} + \gamma G_{t+1:T} \qquad G_{T:T} = 0}$$

Therefore, a **single backward pass** through the rewards computes all returns:

```
def compute_returns(rewards, gamma):
```

```
    # Convert the reward sequence to a NumPy array of floats
    rewards = np.asarray(rewards, dtype=float)

    # Allocate one return for each time step in the trajectory
    returns = np.zeros(len(rewards), dtype=float)

    # At the end of the episode, there are no future rewards
    G = 0.0

    # Process the trajectory backwards, from the last reward to the first

    for t in reversed(range(len(rewards))):

        # Compute the return recursively:
        # G_t = R_{t+1} + gamma * G_{t+1}
        G = rewards[t] + gamma * G

        # Store the return associated with the state visited at time t
        returns[t] = G

    # Return the complete sequence of Monte Carlo returns
    return returns
```

The discount factor controls how strongly distant rewards contribute to the return: a reward received k steps into the future is weighted by γ^k. With $\gamma = 1$, all rewards contribute equally, decreasing γ progressively reduces the influence of distant rewards:

```
import matplotlib.pyplot as plt

# Number of steps into the future
k = np.arange(500)

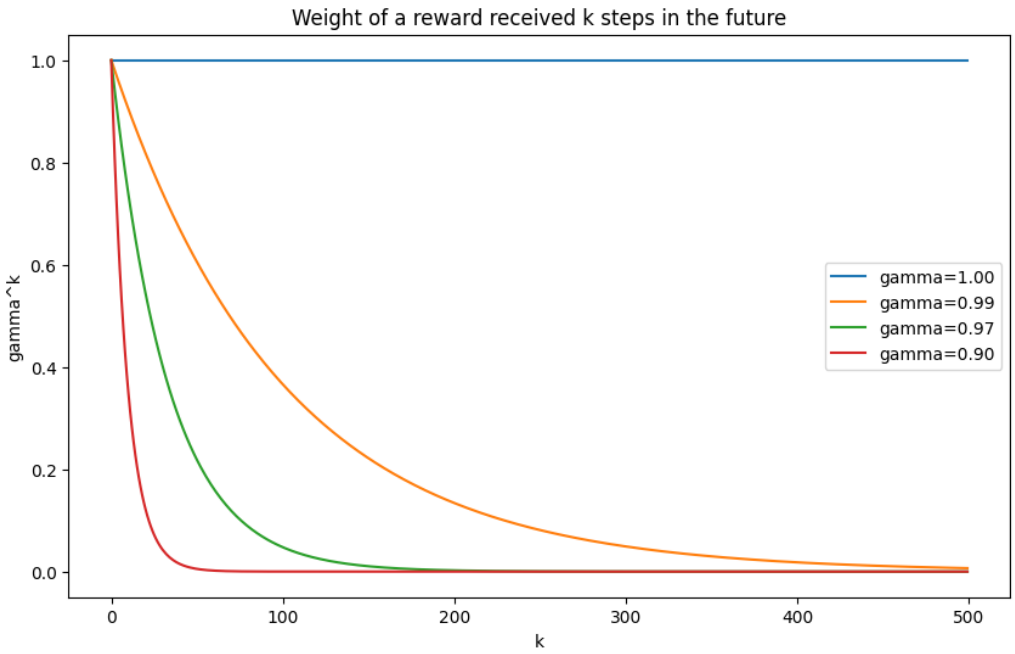
# Plot the weight of future rewards for different discount factors
plt.figure(figsize=(10, 6))

plt.plot(1.00 ** k, label='gamma=1.00')
plt.plot(0.99 ** k, label='gamma=0.99')
plt.plot(0.97 ** k, label='gamma=0.97')
plt.plot(0.90 ** k, label='gamma=0.90')

# Add labels and legend
plt.title('Weight of a reward received k steps in the future')
```

```
plt.xlabel('k')
plt.ylabel('gamma^k')
plt.legend()

plt.show()
```



Now suppose that at time i we visit state $S_i = s$. The corresponding return $G_{i:T}$ is one sample of the return from state s. After many visits to s, we obtain:

$G^{(1)}(s), G^{(2)}(s), \ldots, G^{(N(s))}(s)$

Monte Carlo prediction estimates the expected return by taking their empirical mean:

$$V(s) = \frac{1}{N(s)} \sum_{j=1}^{N(s)} G^{(j)}(s)$$

As the number of samples increases, this estimate approaches the true state value:

$N(s) \to \infty \quad \Rightarrow \quad V(s) \to v_\pi(s)$

This is the fundamental idea behind Monte Carlo prediction: **experience complete returns and average them to estimate their expectation**. We do not need to store all previous returns. The same sample mean can be updated incrementally whenever a new return becomes available:

$$V(S_i) \leftarrow V(S_i) + \frac{1}{N(S_i)} [G_{i:T} - V(S_i)]$$

More generally, we can replace $1/N(S_i)$ with a learning rate α:

$$V(S_i) \leftarrow V(S_i) + \alpha \underbrace{\left[\underbrace{G_{i:T}}_{\text{MC target}} - V(S_i) \right]}_{\text{MC error}}$$

$$\boxed{V(S_i) \leftarrow V(S_i) + \alpha \left[G_{i:T} - V(S_i) \right]}$$

The update therefore follows the familiar pattern:

new estimate = old estimate + step size × error

Notice that **MC target** contains **only rewards actually experienced** by the agent. It uses neither a model of the environment nor an estimate of future value. The price is that the complete return is known only after the episode terminates, so **Monte Carlo methods can update their estimates only after observing the final outcome**.

1.1.3 First-visit and Every-visit Monte Carlo

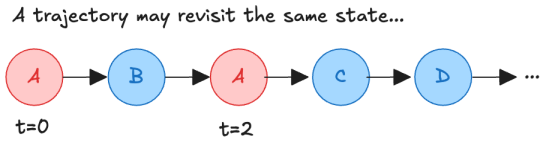
There is one detail we must clarify before turning Monte Carlo prediction into an algorithm. **A single trajectory may visit the same state more than once**. Suppose, for example, that an episode contains:

$S_0 = A, \quad S_1 = B, \quad S_2 = A, \quad S_3 = C, \quad S_4 = D, \ldots$

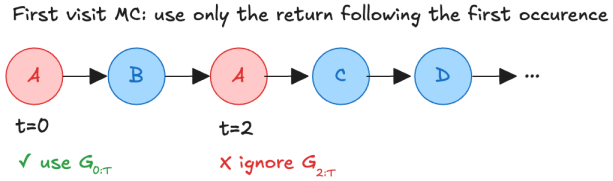
State A is visited twice, and the two visits generally produce different returns:

$G_{0:T} \neq G_{2:T}$

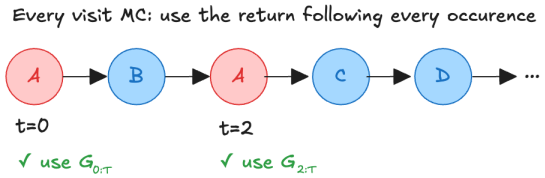
Should both returns be used to update V(A)?



In **first-visit Monte Carlo (FVMC)**, only the return following the **first occurrence** of each state in an episode is used. In this example, A contributes only $G_{0:T}$. Therefore, each state contributes at most one target per episode:



In **every-visit Monte Carlo (EVMC)**, instead, the return following **every occurrence** of a state is used. Therefore, both $G_{0:T}$ and $G_{2:T}$ contribute to the estimate of V(A):



Both approaches are valid and converge under standard conditions. First-visit MC has a particularly direct convergence argument: returns following first visits to a state behave as independent samples with expectation $v_\pi(s)$, so their average converges to the true value by the law of large numbers. Every-visit MC also converges, although returns generated by repeated visits within the same episode are generally correlated. We now implement first-visit Monte Carlo prediction:

```python
def mc(pi, env, gamma=1.0, alpha=0.1, n_episodes=500, max_steps=200):

    # Initialize the current estimate of the state-value function
    v = np.zeros(env.observation_space, dtype=float)

    # Store the value estimates after each episode
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float)

    # Generate and process one complete trajectory per episode
    for e in range(n_episodes):

        # Generate an episode by following the policy
        trajectory = generate_trajectory(pi, env, max_steps)

        # Compute the return associated with every state visit
        # using a single backward pass through the rewards
        returns = compute_returns(trajectory[:, 2], gamma)

        # Track which states have already been processed in this episode
        visited = np.zeros(env.observation_space, dtype=bool)

        # Process the trajectory from beginning to end
        for t, (state, _, _, _, _) in enumerate(trajectory):

            # First-visit MC: ignore repeated visits to the same state
            if visited[state]:
                continue
```

```
            visited[state] = True

            # Monte Carlo target and error
            mc_target = returns[t]
            mc_error = mc_target - v[state]

            # Update the value estimate
            v[state] = v[state] + alpha * mc_error

        # Save the value function after the complete episode
        v_track[e] = v

    return v.copy(), v_track
```

We can now apply MC prediction to the Random Walk environment:

```
v_mc, v_mc_track = mc(pi, random_walk, n_episodes=500)

print('State   MC estimate   True value')
print('---------------------------------')
for s in range(1, 6):
    print(f'{s:^5}   {v_mc[s]:^11.3f}   {random_walk.v_true[s]:^10.3f}')
```

```
State   MC estimate   True value
---------------------------------
  1        0.114         0.167
  2        0.314         0.333
  3        0.432         0.500
  4        0.673         0.667
  5        0.891         0.833
```

To understand how learning evolves, we can plot the estimates of the five non-terminal states across episodes:

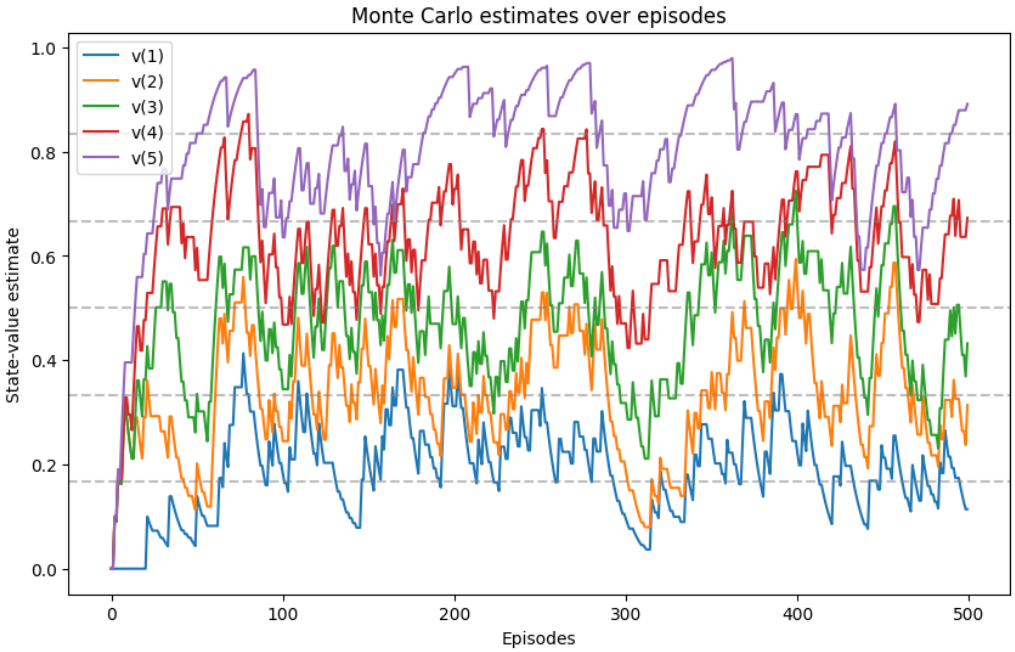
```
plt.figure(figsize=(10, 6))

# Plot the learned value estimates
plt.plot(v_mc_track[:, 1:6])
```

```
# Plot the true state values
plt.axhline(random_walk.v_true[1], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[2], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[3], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[4], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[5], color='gray', linestyle='--', alpha=0.5)

plt.title('Monte Carlo estimates over episodes')
plt.xlabel('Episodes')
plt.ylabel('State-value estimate')
plt.legend(['v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)'])

plt.show()
```



The estimates gradually move toward the true state values. However, **the trajectories are quite noisy**: even after approaching the correct values, they **continue to fluctuate** from episode to episode. This is a characteristic property of Monte Carlo learning. **Each update is based on a complete sampled return, and complete returns can vary considerably across trajectories**. MC targets are **unbiased samples** of the expected return, but they can have **high variance**. To obtain a more reliable view of learning performance, we repeat the experiment many times and measure the root mean-squared error (RMSE) between the estimated and true value functions. For one run, the RMS error after an episode is:

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \left(V(s) - v_\pi(s)\right)^2}$$

We then average this quantity across multiple independent runs:

```
def run_experiment(algorithm, runs, env, pi):

    v_runs = []

    # Repeat the complete learning experiment several times
    for _ in range(runs):
        _, v_track = algorithm(pi, env)
        v_runs.append(v_track)

    v_runs = np.array(v_runs)

    # Remove the two terminal states
    v_runs = v_runs[:, :, 1:-1]

    # Compute the error with respect to the true value function
    error_to_true = v_runs - env.v_true[1:-1]

    # RMS error over the five non-terminal states
    squared_error = np.power(error_to_true, 2)
    mean_squared_error = np.mean(squared_error, axis=-1)
    root_mean_squared_error = np.sqrt(mean_squared_error)

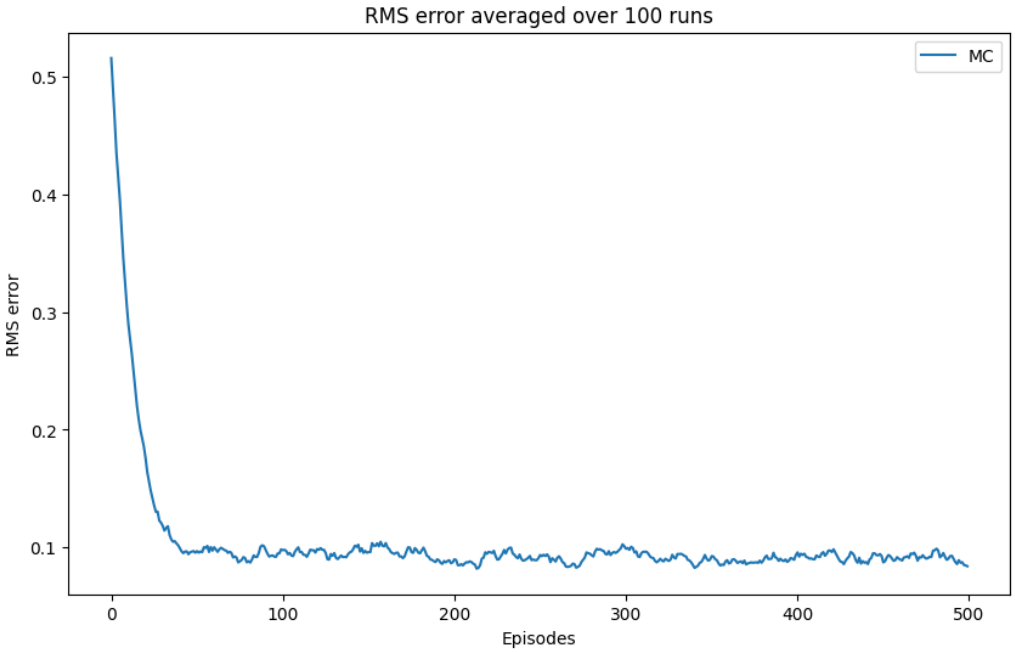
    # Average the learning curve over all independent runs
    rmse_avg_over_runs = np.mean(root_mean_squared_error, axis=0)

    return rmse_avg_over_runs
```

We can now estimate the Monte Carlo learning curve over 100 independent runs:

```
rmse_mc = run_experiment(mc, 100, random_walk, pi)

plt.figure(figsize=(10, 6))
plt.plot(rmse_mc, label='MC')
plt.title('RMS error averaged over 100 runs')
plt.xlabel('Episodes')
plt.ylabel('RMS error')
plt.legend()
plt.show()
```



The learning curve shows that the error decreases rapidly at the beginning, as the agent collects return samples and improves its initial estimates. Later, the improvement becomes slower and the curve settles around a small residual error. The remaining fluctuations are not surprising. Monte Carlo updates use complete returns sampled from entire trajectories, so two visits to the same state can generate substantially different targets depending on what happens later in the episode. This gives MC relatively high-variance updates. This observation motivates the next question: **do we really need to wait until the end of the episode and use the entire sampled return?** Temporal-difference learning) will answer this by replacing part of the return with an existing value estimate.

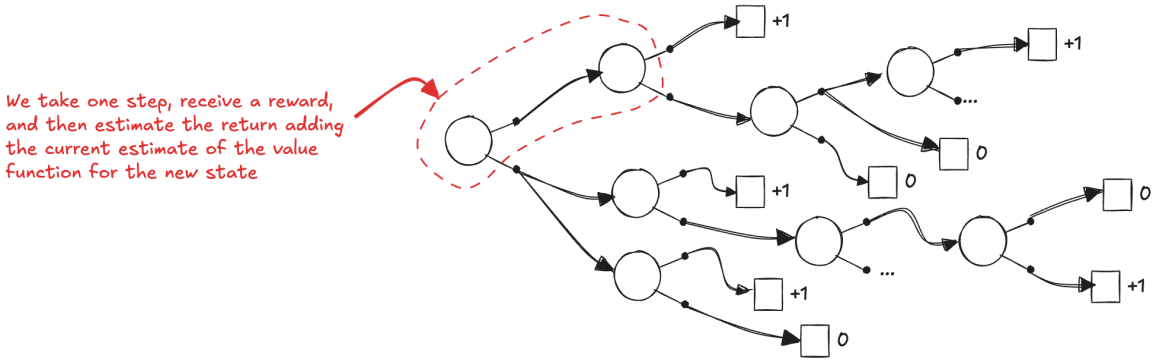
1.1.4 Improving estimates after each step: Temporal-Difference Learning

Monte Carlo prediction has an appealing property: its target is the actual return experienced by the agent. For a fixed policy, this return is an **unbiased sample** of the value we want to estimate. However, this comes with two important limitations. First, MC must **wait until the episode terminates**. The complete return:

$$G_{t:T} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T$$

is not known until all subsequent rewards have been observed. Therefore, the agent cannot update the value function immediately after leaving a state. Second, **complete returns can have high variance**. From time t to termination, the trajectory may contain many stochastic transitions and rewards. All this randomness is accumulated into a single return $G_{t:T}$. Thus, although

the MC target is unbiased, individual samples can differ substantially, producing noisy updates. Can we learn from a transition without waiting for the complete return?



Recall the recursive relationship between consecutive returns:

$$G_{t:T} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T$$

$$= R_{t+1} + \gamma \left(R_{t+2} + \gamma R_{t+3} + \cdots + \gamma^{T-t-2} R_T \right)$$

and therefore:

$$G_{t:T} = R_{t+1} + \gamma G_{t+1:T}$$

Monte Carlo waits until the end of the episode to observe $G_{t+1:T}$. But **we are already learning an estimate of its expectation**: $V(S_{t+1})$. Temporal-Difference learning makes the crucial substitution:

$$G_{t+1:T} \longrightarrow V(S_{t+1})$$

giving the **one-step TD target**:

$$\boxed{G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1})}$$

Instead of waiting for the complete future, **TD uses one reward from experience and then its current estimate of the remaining return**.The value estimate can therefore be updated immediately after each transition:

$$\boxed{V(S_t) \leftarrow V(S_t) + \alpha \underbrace{\left[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t) \right]}_{\text{TD error}}}$$

This is the same learning structure we used for Monte Carlo:

new estimate = old estimate + step size × error

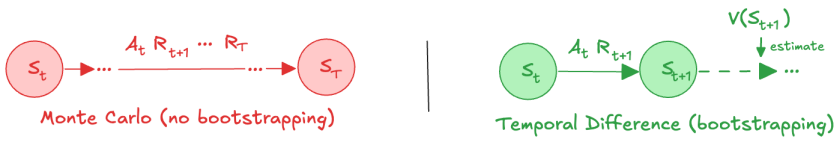
but the target has changed. MC uses the complete action return:

$G_{t:T}$

whereas TD(0) uses the one-step estimated return:

$R_{t+1} + \gamma V(S_{t+1})$

This idea of updating an estimate using another estimate is called **bootstrapping**:



The observed reward R_{t+1} **anchors the update in real experience**, while $V(S_{t+1})$ supplies an estimate of everything that may happen afterward. There is an important trade-off. The MC target contains the randomness accumulated over the entire remainder of the trajectory, giving it potentially high variance but no bootstrapping bias. The TD target depends only on the observed one-step transition before bootstrapping, so it typically **has lower variance**, but its target depends on the current, possibly inaccurate estimate $V(S_{t+1})$ and therefore **introduces bias**.This also explains why TD can be viewed as combining ideas from Monte Carlo and Dynamic Programming. Like MC, TD learns directly from sampled experience and requires no model of the environment. Like DP, TD bootstraps, updating one value estimate using another value estimate. The crucial difference from DP is that DP uses an expectation over all possible one-step transitions, whereas TD learns from the single transition actually sampled from the environment. We can implement one-step TD, usually called TD(0), directly:

```python
def td(pi, env, gamma=1.0, alpha=0.1, n_episodes=500):

    # Initialize the state-value estimates
    v = np.zeros(env.observation_space, dtype=float)

    # Store the value estimates after each episode
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float)

    # Run one interaction sequence for each episode
    for e in range(n_episodes):

        # Start a new episode
        state = env.reset()
        done = False

        # TD learns after every transition
        while not done:

            # Select an action according to the policy
```

```
            action = pi(state)

            # Take one step in the environment
            next_state, reward, terminated, truncated, info = env.step(action)
            done = terminated or truncated

            # Compute the one-step TD target and TD error
            td_target = reward + gamma * v[next_state] * (not done)
            td_error = td_target - v[state]

            # Immediately update the current state
            v[state] = v[state] + alpha * td_error

            # Continue from the next state
            state = next_state

        # Store the estimates at the end of the episode
        v_track[e] = v

    return v.copy(), v_track
```

Notice the fundamental difference from the MC implementation: there is **no trajectory to store** and **no return to compute** after termination. Each experience:

$(S_t, A_t, R_{t+1}, S_{t+1})$

is sufficient to perform an update immediately. We can now apply TD(0) to the Random Walk environment:

```
v_td, v_td_track = td(pi, random_walk, n_episodes=500)

print('State   TD estimate   True value')
print('---------------------------------')
for s in range(1, 6):
    print(f'{s:^5}   {v_td[s]:^11.3f}   {random_walk.v_true[s]:^10.3f}')
```

```
State   TD estimate   True value
---------------------------------
  1        0.158        0.167
  2        0.336        0.333
  3        0.513        0.500
```

```
4          0.662          0.667
5          0.814          0.833
```

and inspect how the estimates evolve:

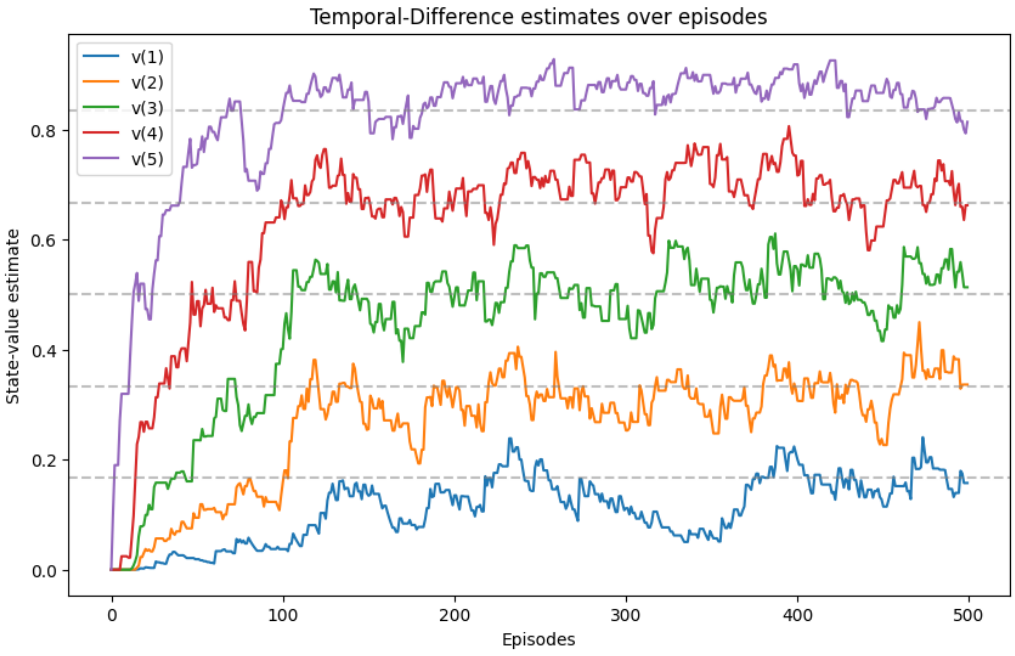
```
plt.figure(figsize=(10, 6))

# Plot the learned value estimates
plt.plot(v_td_track[:, 1:6])

# Plot the true state values
plt.axhline(random_walk.v_true[1], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[2], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[3], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[4], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[5], color='gray', linestyle='--', alpha=0.5)

plt.title('Temporal-Difference estimates over episodes')
plt.xlabel('Episodes')
plt.ylabel('State-value estimate')
plt.legend(['v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)'])

plt.show()
```



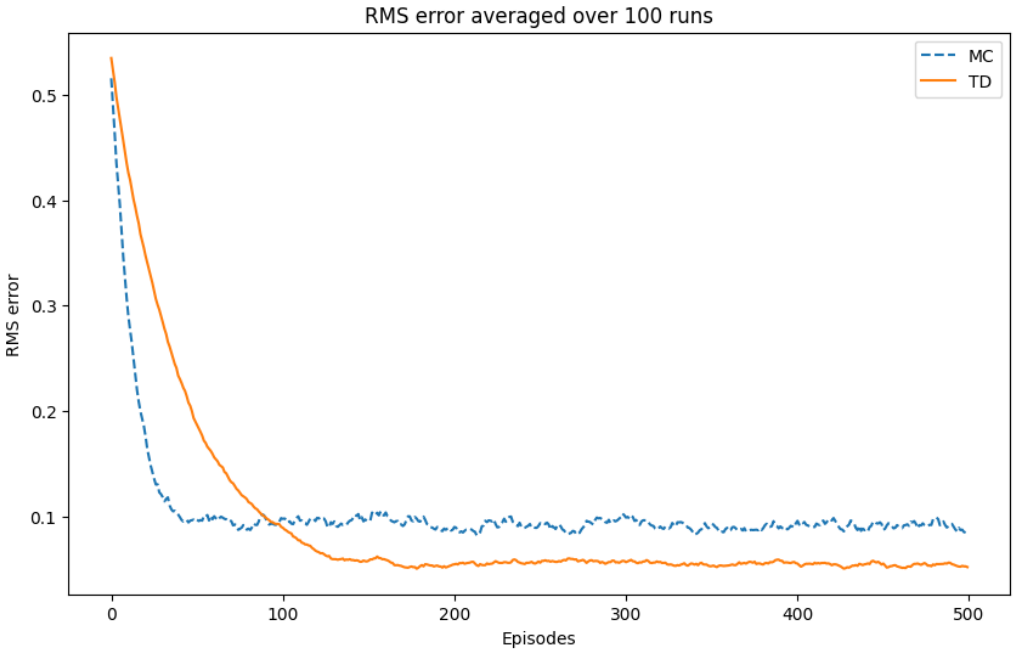
Compared with MC using the same constant step size, the TD estimates typically **fluctuate less** because each target contains much less sampled randomness. However, **early in learning they can be systematically inaccurate** because TD bootstraps from value estimates that are themselves still inaccurate. This is the characteristic **bias–variance trade-off** between the two methods. To compare them more reliably, we average the RMS error over many independent runs:

```
rmse_td = run_experiment(td, 100, random_walk, pi)

plt.figure(figsize=(10, 6))

plt.plot(rmse_mc, linestyle='--', label='MC')
plt.plot(rmse_td, label='TD')

plt.title('RMS error averaged over 100 runs')
plt.xlabel('Episodes')
plt.ylabel('RMS error')
plt.legend()
plt.show()
```

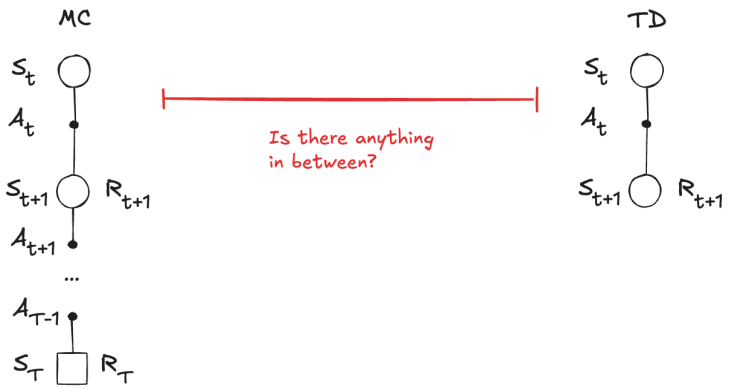


For this Random Walk problem and these hyperparameters, the two learning curves cross. **Monte Carlo initially reduces the error faster, but after some experience TD overtakes it and stabilizes at a lower error**. This behavior provides a clear illustration of the **bias–variance trade-off** between the two methods. Early in learning, the main problem for TD is **bias**. Monte Carlo

learns from the complete outcome of an episode, so the final result can immediately influence every state visited along the trajectory. TD instead bootstraps from its current estimate of the next state. At the beginning, these estimates are themselves inaccurate, so TD is effectively learning from imperfect predictions. Moreover, information about the final reward propagates backward only gradually. In this early phase, Monte Carlo therefore has an advantage: its targets are noisy, but they are grounded entirely in what actually happened. As learning progresses, the situation changes. The value estimates used for bootstrapping become increasingly accurate, so the bias introduced by TD becomes less important. At this point, **variance** becomes the dominant difference between the two methods. Monte Carlo targets still depend on the complete trajectory and therefore accumulate all the randomness encountered until termination. TD targets depend only on one sampled transition before bootstrapping from the learned value function, so they generally have much lower variance. With the constant learning rate used in this experiment, neither method settles exactly at the true values: new experience continues to perturb the estimates. Because Monte Carlo uses higher-variance targets, these perturbations remain larger. TD eventually fluctuates within a narrower region around the true values and consequently achieves a lower residual error. The comparison should therefore not be summarized simply as "TD learns faster than MC". Instead, it shows how the bias–variance trade-off changes during learning: **bootstrapping can hurt early**, when the estimates used as targets are still inaccurate, b**ut it can provide much greater stability later**, once those estimates have become reliable. The exact point at which one method overtakes the other is not fundamental. It depends on the initialization, learning rate, environment, and other experimental choices. What matters is the underlying trade-off: Monte Carlo relies more heavily on real but high-variance experience, whereas TD relies more heavily on lower-variance but initially biased bootstrapped estimates. This naturally raises the next question: **can we choose something between these two extremes**, using several real rewards before bootstrapping? This leads directly to n-step Temporal-Difference learning.

1.1.5 Improving estimates after multiple steps: n-steps TD

Monte Carlo and TD(0) represent two extremes in how we construct a target. Monte Carlo waits until the end of the episode, using all the rewards actually experienced before updating. TD(0) waits only one step, using one observed reward and then bootstrapping from the current value estimate. But is there something in between?



Instead of bootstrapping after one step, we could wait for two steps, three steps, or, more generally, n steps. This gives us a family of methods forming a spectrum between TD(0) and Monte Carlo. This family is called **n-step Temporal-Difference learning**. It allows us to control how many actual rewards we observe before bootstrapping, providing a way **to balance bootstrapping bias and sampling variance**. Starting from state S_t, we interact with the environment for up to n steps:

$S_t, A_t, R_{t+1}, S_{t+1}, \dots, R_{t+n}, S_{t+n}$

If the episode terminates before n transitions have been observed, we simply stop at the terminal state and use the rewards available up to termination. As with MC, we need a helper function to generate trajectory segments, but this time each segment contains at most n steps:

```
def generate_path(pi, env, state, n_steps):

    # Store the experiences collected along the path
    path = []

    done = False

    # Collect at most n transitions
    while len(path) < n_steps:

        # Select an action according to the policy
        action = pi(state)

        # Take one step in the environment
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated

        # Store the experience
```

```
        path.append((state, action, reward, next_state, done))

        # Stop early if the episode terminates
        if done:
            break

        # Continue from the next state
        state = next_state

    return np.array(path, dtype=object)
```

The three cases can now be compared directly. Monte Carlo uses the complete return:

$$G_{t:T} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T$$
$$V(S_t) \leftarrow V(S_t) + \alpha \left[G_{t:T} - V(S_t) \right]$$

One-step TD instead uses one actual reward and then bootstraps:

$$G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1})$$
$$V(S_t) \leftarrow V(S_t) + \alpha \left[G_{t:t+1} - V(S_t) \right]$$

The n-step method extends exactly the same idea. We observe n actual rewards and then bootstrap from the value of the state reached after those n steps:

$$\boxed{G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})}$$

$$\boxed{V(S_t) \leftarrow V(S_t) + \alpha \underbrace{\left[\underbrace{G_{t:t+n}}_{\text{n-step TD target}} - V(S_t) \right]}_{\text{n-step TD error}}}$$

The interpretation is straightforward: the first n rewards come from actual experience, everything beyond S_{t+n} is summarized by the current estimate $V(S_{t+n})$. As n increases, we use more real rewards and rely less on bootstrapping. At one extreme:

$$n = 1 \quad \Rightarrow \quad G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1}) \quad \Rightarrow \quad \text{TD}(0)$$

At the other extreme, if the episode terminates before the n-th step, there is no future value from which to bootstrap:

$$t + n \geq T \quad \Rightarrow \quad G_{t:t+n} = G_{t:T} \quad \Rightarrow \quad \text{Monte Carlo}$$

Thus, TD(0) and Monte Carlo are the two extremes of the same family of n-step methods.

```python
def ntd(pi, env, gamma=1.0, n_steps=3, alpha=0.1, n_episodes=500):

    # Initialize the state-value estimates
    v = np.zeros(env.observation_space, dtype=float)

    # Store the estimates after each episode
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float)

    # Run one complete episode at a time
    for e in range(n_episodes):

        state = env.reset()
        done = False

        # Process the episode in paths of at most n steps
        while not done:

            # Generate the next path
            path = generate_path(pi, env, state, n_steps)

            # The path may be shorter than n if termination occurs
            n = len(path)

            # Compute the partial returns from the observed rewards
            partial_returns = compute_returns(path[:, 2], gamma)

            # Determine where to bootstrap
            bootstrap_state = path[n - 1, 3]
            path_done = path[n - 1, 4]

            # Update the states encountered along the path
            for t, (state, _, _, next_state, done) in enumerate(path):

                # Add the bootstrapped value after the observed rewards
                ntd_target = (
                    partial_returns[t]
                    + (gamma ** (n - t))
                    * v[bootstrap_state]
                    * (not path_done)
                )

                ntd_error = ntd_target - v[state]
                v[state] = v[state] + alpha * ntd_error
```

```
                state = next_state

        # Store the value function after the episode
        v_track[e] = v

    return v.copy(), v_track
```

We can apply the method to the Random Walk environment using 3-step TD:

```
v_ntd, v_ntd_track = ntd(
    pi,
    random_walk,
    n_steps=3,
    n_episodes=500
)

print('State   nTD estimate   True value')
print('---------------------------------')
for s in range(1, 6):
    print(f'{s:^5}   {v_ntd[s]:^12.3f}   {random_walk.v_true[s]:^10.3f}')
```

```
State   nTD estimate   True value
---------------------------------
  1         0.129          0.167
  2         0.305          0.333
  3         0.463          0.500
  4         0.695          0.667
  5         0.835          0.833
```

We can also visualize how the estimates evolve during learning:

```
plt.figure(figsize=(10, 6))

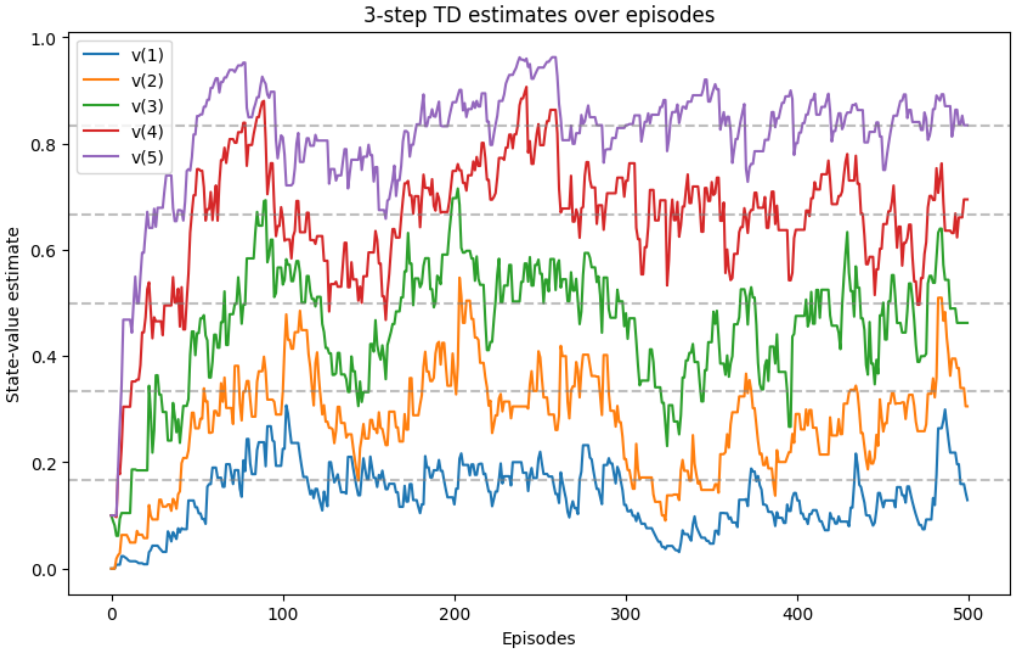
plt.plot(v_ntd_track[:, 1:6])

# Plot the true state values
plt.axhline(random_walk.v_true[1], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[2], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[3], color='gray', linestyle='--', alpha=0.5)
```

```
plt.axhline(random_walk.v_true[4], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[5], color='gray', linestyle='--', alpha=0.5)

plt.title('3-step TD estimates over episodes')
plt.xlabel('Episodes')
plt.ylabel('State-value estimate')
plt.legend(['v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)'])

plt.show()
```



Finally, we compare MC, TD(0), and 3-step TD using the same step size:

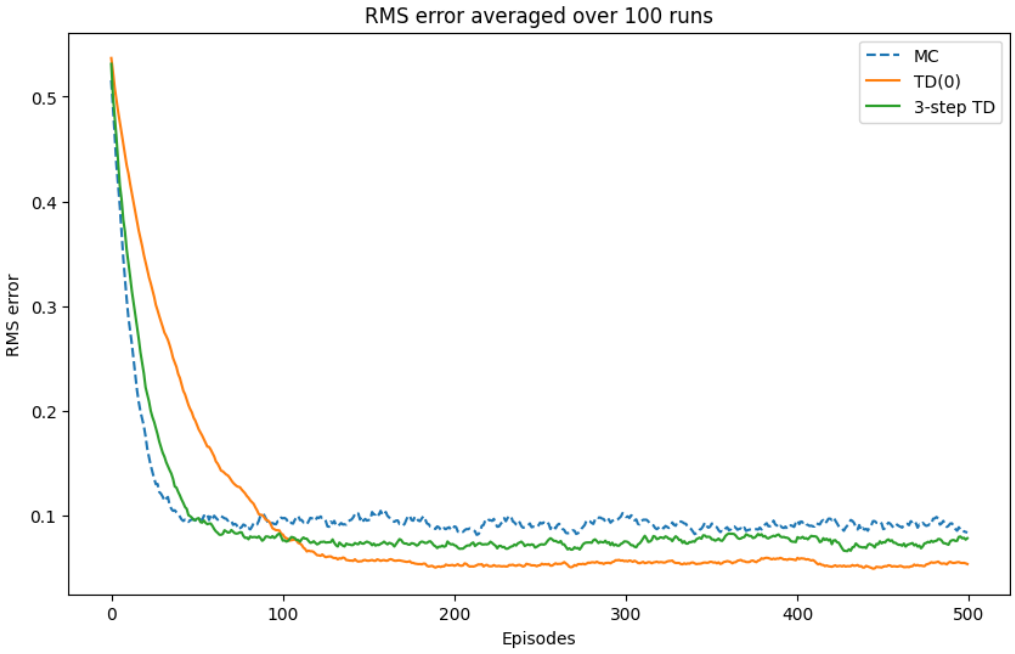
```
rmse_ntd = run_experiment(ntd, 100, random_walk, pi)

plt.figure(figsize=(10, 6))

plt.plot(rmse_mc, linestyle='--', label='MC')
plt.plot(rmse_td, label='TD(0)')
plt.plot(rmse_ntd, label='3-step TD')

plt.title('RMS error averaged over 100 runs')
plt.xlabel('Episodes')
plt.ylabel('RMS error')
plt.legend()
```

```
plt.show()
```



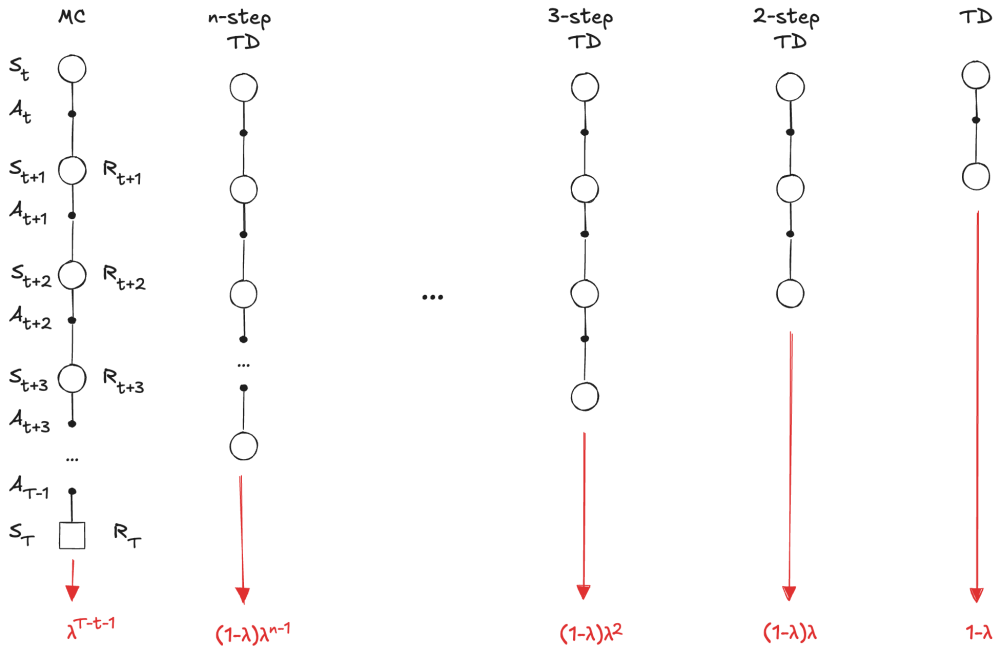
The parameter n determines **how far we rely on real experience before bootstrapping**. With a small n, the target contains few sampled rewards and relies heavily on the current value estimates. This generally gives lower variance but greater dependence on potentially inaccurate estimates. With a larger n, the target contains more actual rewards and relies less on bootstrapping, reducing this source of bias but accumulating more randomness from the trajectory. Thus, **n controls a bias–variance trade-off**:

small $n \longrightarrow$ more bootstrapping, lower variance
large $n \longrightarrow$ less bootstrapping, higher variance

There is no universally optimal value of n. It depends on the environment, the reward structure, the step size, and the learning problem. In fact, intermediate values of n can perform better than both TD(0) and Monte Carlo, as shown in the classical Random Walk experiments.

1.1.6 Improving estimates of all visited states: TD(λ)

The n-step TD introduces a new question: **what is a good value of n?** How many real rewards should we observe before bootstrapping? Instead of choosing a single n, we can take a different approach: **combine all the n-step targets into a single target**, giving more weight to shorter returns and progressively less weight to longer ones.



The parameter

$\lambda \in [0, 1]$

controls how much weight is given to short versus long returns:

$\lambda = 0 \quad \Rightarrow \quad \text{TD}(0)$
$0 < \lambda < 1 \quad \Rightarrow \quad$ mixture of multiple n-step targets
$\lambda = 1 \quad \Rightarrow \quad$ Monte Carlo behavior

Recall the sequence of targets:

$$G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1})$$
$$G_{t:t+2} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$$
$$G_{t:t+3} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 V(S_{t+3})$$
$$\vdots$$
$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

until the complete Monte Carlo return:

$$G_{t:T} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T$$

TD(λ) combines these targets using **exponentially decaying weights**:

$$\boxed{G^{\lambda}_{t:T} = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{T-t-1} G_{t:T}}$$

The weights are therefore:

$1-\lambda, \quad (1-\lambda)\lambda, \quad (1-\lambda)\lambda^2, \quad \ldots$

for the 1-step, 2-step, 3-step, ... targets.

Their meaning is simple: shorter targets receive more weight, while longer targets receive exponentially less weight. The parameter controls **how quickly that decay happens**. Small value concentrates most of the weight on short-horizon targets, so the method behaves more like TD(0). Large values spread more weight toward long-horizon targets, so the method behaves more like Monte Carlo. The Monte Carlo term is different:

$\lambda^{T-t-1} G_{t:T}$

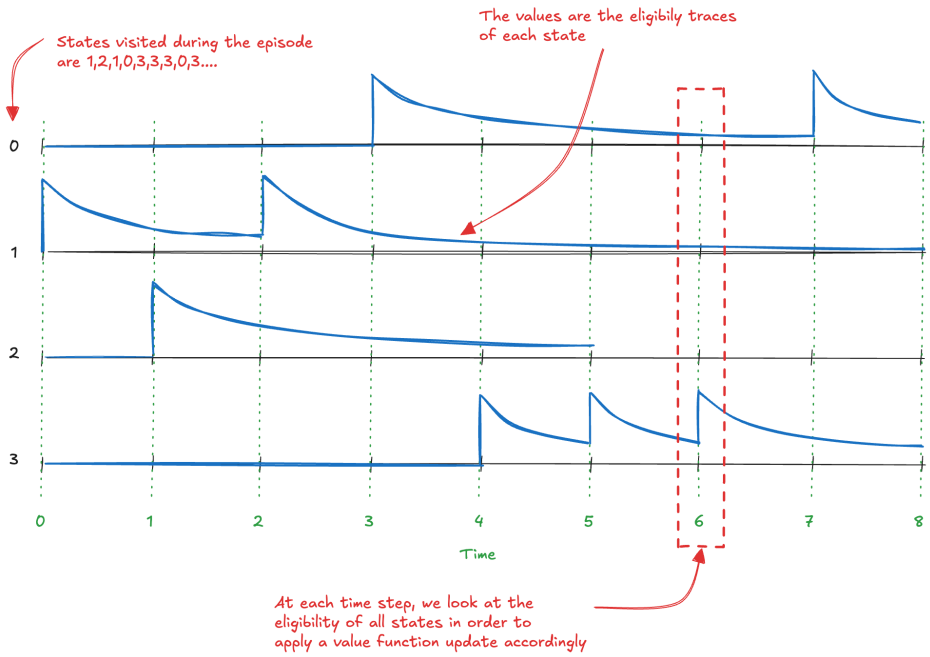
because it receives **all the remaining weight once the episode terminates**. There is no later target beyond the terminal state, so the residual probability mass that would otherwise continue along the geometric sequence is assigned to the complete return. This is what makes all weights sum to one. The value estimate is then updated toward the λ-return:

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{\left[\underbrace{G^{\lambda}_{t:T}}_{\text{TD}(\lambda)\text{ target}} - V(S_t) \right]}_{\text{TD}(\lambda)\text{ error}}$$

This is the **forward view**: from each visited state, we look forward, construct all possible n-step targets, and combine them into a single target. The **limitation is practical**: to compute the complete return, we need to know how the episode ends. In this offline form, we must therefore **wait until termination before performing the update**. The next question is therefore: can we obtain the same effect online, one transition at a time? This leads to the backward view and eligibility traces.

1.1.7 The Backward View: Eligibility Traces

The practical implementation uses a **backward view** based on **eligibility traces**. Instead of standing at state S_t and asking which future rewards should affect it, we reverse the perspective. At every new transition, we compute the ordinary one-step TD error and ask:** which previously visited states should receive credit for this new information?** An eligibility trace is a short-term memory associated with each state. Recently visited states have high eligibility; as time passes, their eligibility progressively decays.



At the beginning of every episode, all traces are initialized to zero:

$E(s) = 0 \qquad \forall s$

We then interact with the environment for one step:

$S_t, A_t, R_{t+1}, S_{t+1}$

When state S_t is visited, we increase its eligibility:

$E(S_t) \leftarrow E(S_t) + 1$

This is called an **accumulating trace**: if the same state is visited repeatedly, its eligibility can become larger than one. We then compute the usual one-step TD error:

$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$

The crucial difference from TD(0) is what we do with this error. TD(0) uses δ_t to update only the current state. TD(λ) propagates the same TD error backward to all eligible states:

$\boxed{V \leftarrow V + \alpha\, \delta_t E}$

Here V and E are vectors. Therefore, every state with a non-zero eligibility trace receives part of the update:

$V(s) \leftarrow V(s) + \alpha\, \delta_t E(s)$

Recently visited states receive a larger update because their eligibility is larger, states visited further in the past receive progressively less credit. After the update, the traces decay:

$E \leftarrow \gamma\lambda E$

This equation reveals the role of λ particularly clearly. A small value makes traces disappear quickly, so TD errors are propagated only a short distance into the past. A large value keeps states eligible for longer, allowing new information to propagate much further backward along the trajectory. The complete interaction therefore follows a simple cycle:

visit state → increase eligibility → compute TD error → update eligible states → decay traces

This is the **backward view** of TD(λ). Rather than explicitly constructing many future n-step returns, it uses the current TD error together with a fading memory of past states. We can implement accumulating-trace as follows:

```python
def td_lambda(pi, env, gamma=1.0, lambda_=0.3,
              alpha=0.1, n_episodes=500):

    # Initialize the state-value estimates
    v = np.zeros(env.observation_space, dtype=float)

    # Store the estimates after each episode
    v_track = np.zeros((n_episodes, env.observation_space), dtype=float)

    # Eligibility trace associated with every state
    E = np.zeros(env.observation_space, dtype=float)

    # Run one complete episode at a time
    for e in range(n_episodes):

        # Eligibility traces are reset at the beginning of each episode
        E.fill(0)

        state = env.reset()
        done = False

        while not done:

            # Select an action according to the policy
            action = pi(state)

            # Interact with the environment for one step
            next_state, reward, terminated, truncated, info = env.step(action)
            done = terminated or truncated
```

```
            # Compute the ordinary one-step TD target and error
            td_target = reward + gamma * v[next_state] * (not done)
            td_error = td_target - v[state]

            # Make the current state eligible for credit
            E[state] = E[state] + 1

            # Propagate the current TD error to all eligible states
            v = v + alpha * td_error * E

            # Decay the eligibility traces
            E = gamma * lambda_ * E

            # Continue from the next state
            state = next_state

        # Store the value estimates after the episode
        v_track[e] = v

    return v.copy(), v_track
```

Notice the key difference from ordinary TD(0). The TD error is still computed from one transition:

$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$

but that error is no longer used only for S_t. Through the eligibility traces, one TD error can update many states visited earlier in the trajectory. This is why TD(λ) can propagate information much more rapidly through a sequence of states while still operating online. We can apply it to the Random Walk environment:

```
v_td_lambda, v_td_lambda_track = td_lambda(
    pi,
    random_walk,
    lambda_=0.3,
    n_episodes=500
)

print('State   TD(lambda)   True value')
print('---------------------------------')
for s in range(1, 6):
    print(f'{s:^5}   {v_td_lambda[s]:^10.3f}   {random_walk.v_true[s]:^10.3f}')
```

```
State   TD(lambda)   True value
--------------------------------
  1       0.075        0.167
  2       0.195        0.333
  3       0.344        0.500
  4       0.535        0.667
  5       0.680        0.833
```

and visualize the evolution of the estimates:

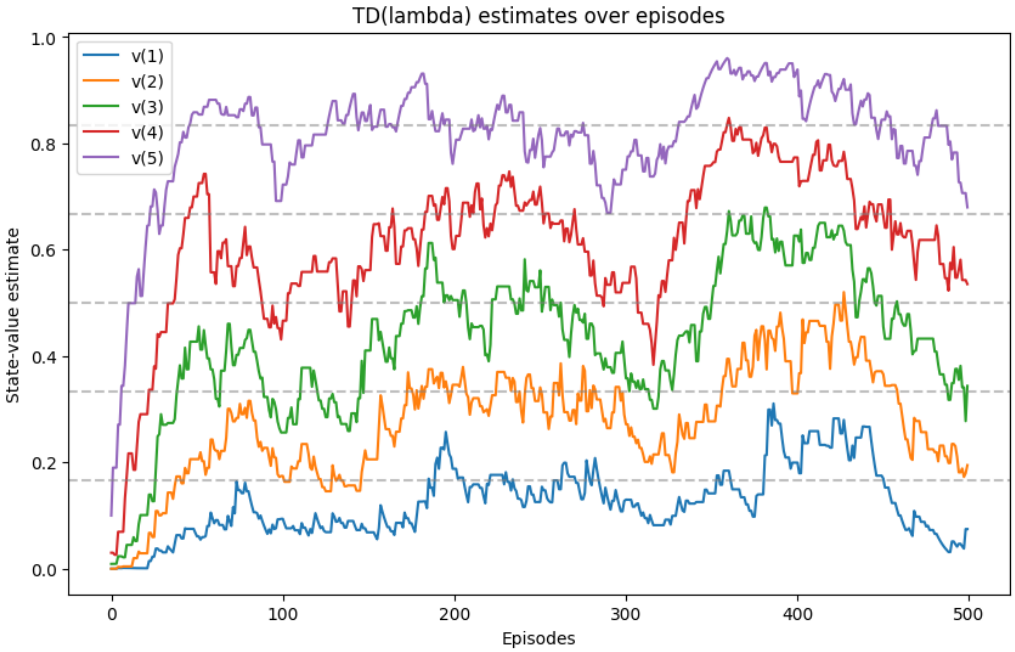
```
plt.figure(figsize=(10, 6))

plt.plot(v_td_lambda_track[:, 1:6])

# Plot the true state values
plt.axhline(random_walk.v_true[1], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[2], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[3], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[4], color='gray', linestyle='--', alpha=0.5)
plt.axhline(random_walk.v_true[5], color='gray', linestyle='--', alpha=0.5)

plt.title('TD(lambda) estimates over episodes')
plt.xlabel('Episodes')
plt.ylabel('State-value estimate')
plt.legend(['v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)'])

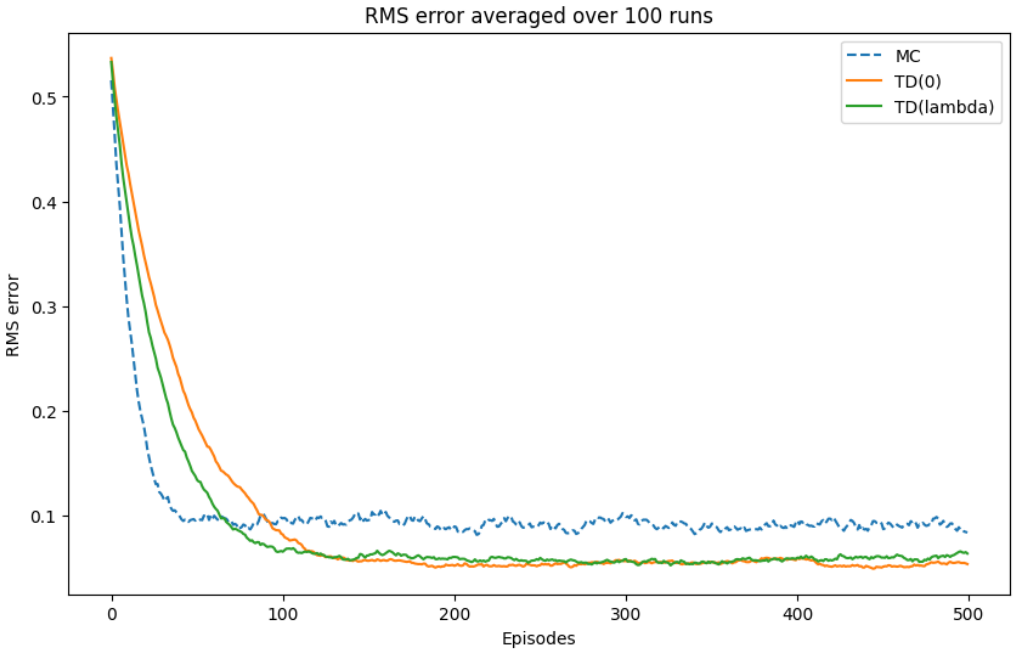
plt.show()
```



Finally, we can compare its learning curve with MC and TD(0):

```
rmse_td_lambda = run_experiment(td_lambda, 100, random_walk, pi)

plt.figure(figsize=(10, 6))
plt.plot(rmse_mc, linestyle='--', label='MC')
plt.plot(rmse_td, label='TD(0)')
plt.plot(rmse_td_lambda, label='TD(lambda)')
plt.title('RMS error averaged over 100 runs')
plt.xlabel('Episodes')
plt.ylabel('RMS error')
plt.legend()
plt.show()
```



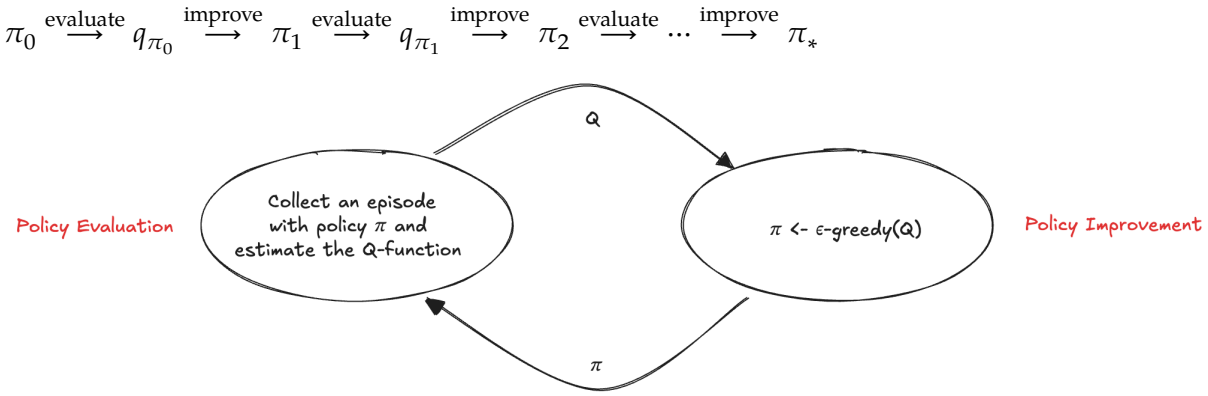
For the particular Random Walk experiment and the selected hyperparameters, TD(λ) may achieve a lower error than either MC or TD(0). The important conclusion, however, is not that it is universally better. Rather, the methos gives us a **continuous mechanism for controlling how far credit propagates through time**. Increasing λ allows each TD error to affect states progressively further into the past. At $\lambda = 1$, the method approaches Monte Carlo behavior while retaining an incremental, online implementation. This is the key conceptual advantage of eligibility traces: instead of choosing a single temporal horizon$n, TD(\lambda$) combines information across multiple horizons and efficiently assigns new information backward to previously visited states.

1.2 Policy Improvement

So far, we have focused on policy evaluation: given a fixed policy, how good is it? We answered this question by estimating its value function, which tells us the expected return from each state. But evaluating a policy does not improve it. Once we know how good different choices are, we need to use that information to make better decisions. The basic idea of policy improvement is to make the policy greedier with respect to the current value estimates. In each state, we prefer the action with the highest action value:

$$\boxed{\pi_{k+1}(s) = \arg\max_a q_{\pi_k}(s,a)}$$

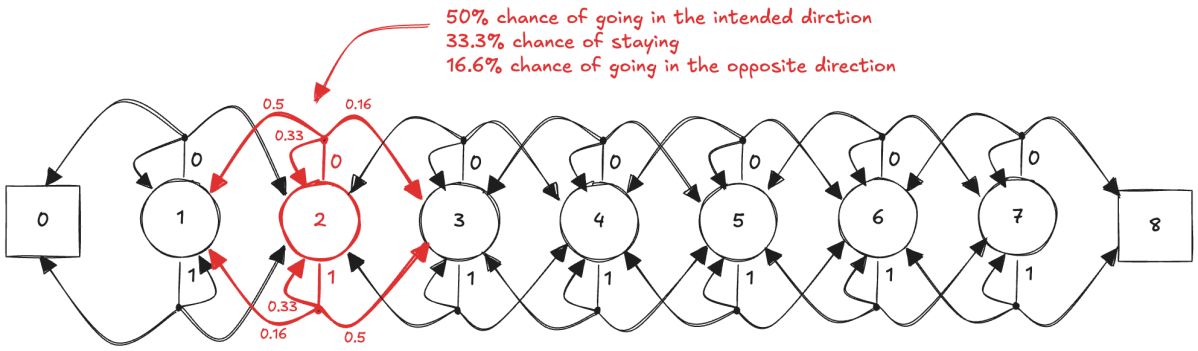
This is the same principle used in Policy Iteration with Dynamic Programming: evaluate the current policy, improve it using the resulting values, evaluate the new policy, and repeat:



There is, however, a **fundamental difference**. In Dynamic Programming, we could evaluate and improve policies using a known model of the environment. In reinforcement learning, the model is unknown: **values must instead be learned from sampled experience**. The two processes therefore interact continuously: evaluation learns how good the current behavior is, while improvement uses those estimates to produce better behavior. We do not even need to wait for either process to finish before starting the other. This general interaction between policy evaluation and policy improvement is called **Generalized Policy Iteration (GPI)**. It provides a common architecture for a large class of reinforcement learning algorithms: values are continually adjusted to better predict returns, while the policy is continually adjusted to become better with respect to those values.

1.2.1 Slippery Walk Environment

For policy evaluation, the Random Walk environment was intentionally designed so that the selected action did not affect the transition. This allowed us to focus entirely on learning state values, but it also meant that there was no meaningful policy to improve: every policy produced the same behavior. To study policy improvement, we therefore need **a new environment in which actions actually matter**. We use the **Slippery Walk**, a single-row grid world with seven non-terminal states. The environment is slippery, similarly to Frozen Lake: the effects of actions are stochastic. If the agent chooses to move left, it is more likely to move left, but it may instead move right or remain in the same state. The same uncertainty applies when choosing to move right. The key difference from Random Walk is that **actions influence the transition probabilities**. The environment is still stochastic, so the agent cannot completely control what happens, but its decisions now affect the distribution of future states and rewards.This gives us exactly what we need for the control problem: **different policies can produce different expected returns**, so learning which actions are more valuable can lead to a better policy.




```python
import random
import numpy as np

class SlipperyWalk:

    observation_space = 9
    action_space = 2

    def __init__(self):
        self.state = None
        self.terminated = False
        self.reset()

    def reset(self):
        # Start from a random non-terminal state
        self.state = np.random.choice([1, 2, 3, 4, 5, 6, 7])
        self.terminated = False
        return self.state

    def step(self, action):

        if self.terminated:
            raise ValueError('Episode has already terminated')

        if action not in [0, 1]:
            raise ValueError('Invalid action')

        # Convert the action into the intended direction
        direction = -1 if action == 0 else 1

        # The environment may follow the intended direction,
        # move in the opposite direction, or remain in place
```

```
        true_direction = random.choices(
            [direction, -direction, 0],
            weights=[3, 1, 2],
            k=1
        )[0]

        self.state += true_direction

        reward = 0

        # Left terminal state
        if self.state == 0:
            self.terminated = True

        # Right terminal state
        elif self.state == 8:
            self.terminated = True
            reward = 1

        return self.state, reward, self.terminated, False, {}
```

Since this is a small environment, we can also solve its MDP exactly with Dynamic Programming and use the resulting optimal value function as a reference for our learning algorithms:

```
slippery_walk = SlipperyWalk()

optimal_v = np.array([
    0.0000,
    0.5637,
    0.7630,
    0.8449,
    0.8892,
    0.9220,
    0.9515,
    0.9806,
    0.0000
])
```

The learning algorithms will not have access to these values, we use them only afterward to evaluate how close the learned estimates are to the true optimum.

1.2.2 Improving policies after each episode: Monte Carlo control

If the action-value function of the current policy were known exactly, the **policy-improvement theorem** tells us that choosing greedily with respect to it produces a policy that is at least as good as the current one:

$$q_{\pi_k}(s, \pi_{k+1}(s)) = q_{\pi_k}\left(s, \arg\max_a q_{\pi_k}(s,a)\right) = \max_a q_{\pi_k}(s,a) \geq q_{\pi_k}(s, \pi_k(s)) = v_{\pi_k}(s)$$

This is the principle we use in Dynamic Programming. The crucial difference is that we no longer know the MDP, so **we cannot compute action values exactly**. We must learn them from sampled experience. Two changes are therefore necessary.

First, **we must estimate the action-value function Q(s,a) rather than only the state-value function V(s)**. Without a model, knowing that a state is valuable does not tell us which action leads to that value. By estimating Q(s,a), we can compare the available actions directly. The Monte Carlo update therefore becomes:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \underbrace{\left[\underbrace{G_{t:T}}_{\text{MC target}} - Q(S_t, A_t) \right]}_{\text{MC error}}$$

Second, **the agent must explore**. If it always selects the action currently believed to be best, other actions may never be tried and their values may never be learned. This is particularly problematic in model-free learning: we obtain information only about the state-action pairs that we actually visit. We therefore use an epsilon-greedy behavior policy. Most of the time, the agent exploits its current knowledge by selecting the action with the highest estimated value; with probability ε, it explores by selecting a random action:

```python
def select_action(state, q, epsilon=0.1):

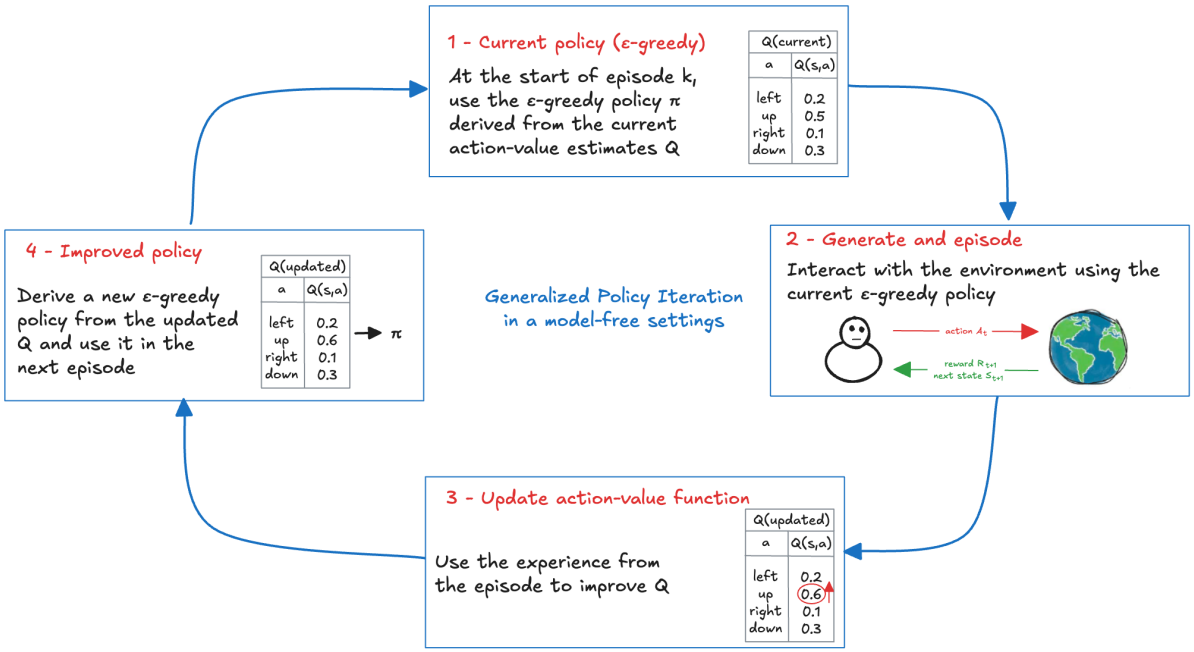
    # Exploit: select the action with the highest estimated value
    if np.random.uniform() > epsilon:
        action = np.argmax(q[state])

    # Explore: select a random action
    else:
        action = np.random.randint(q.shape[1])

    return action
```

Notice that this is not simply "evaluation followed by greedy improvement". Evaluation and improvement are now intertwined: each episode is generated using the current epsilon-greedy

policy, the resulting experience improves Q, and the improved Q immediately changes the policy used to generate future episodes. This is **Generalized Policy Iteration** in a model-free setting:



During learning, we want the agent to **move gradually from exploration toward exploitation**. We therefore decrease ε: initially the agent explores aggressively, while later it increasingly relies on the actions it has learned to value. We also decrease the learning rate α. A large α allows rapid learning from early experience, whereas a smaller α makes the estimates progressively more stable. For these experiments, we use the same **exponentially decaying schedule** for both parameters:

```
def decay_schedule(init_value, min_value, decay_steps, max_steps):

    # Number of steps remaining after the decay phase
    rem_steps = max_steps - decay_steps

    # Generate an exponentially decaying sequence
    values = np.logspace(
        start=0,
        stop=-2,
        num=decay_steps,
        base=10
    )
```

```
    # Normalize the sequence between 0 and 1
    values = (values - values.min()) / (values.max() - values.min())

    # Scale it between the initial and minimum values
    values = (init_value - min_value) * values + min_value

    # Keep the minimum value for the remaining steps
    values = np.pad(values, (0, rem_steps), 'edge')

    return values
```

A technical distinction is useful here. A finite schedule ending at min_epsilon=0.1 does not literally satisfy the asymptotic condition $\varepsilon \to 0$. It is an experimental schedule that preserves some exploration throughout training. If we wanted the theoretical greedy in the limit with infinite exploration (GLIE) property, we would require exploration to persist indefinitely while becoming progressively smaller.

Trajectory generation is also slightly different from the prediction problem. Previously, actions came from a fixed policy. For control, the policy changes as Q changes, and actions are selected using the current epsilon-greedy strategy.

```
def generate_trajectory_control(select_action, q, env, epsilon, max_steps=200):

    # Store all experiences collected during the episode
    trajectory = []

    # Start a new episode
    state = env.reset()

    for _ in range(max_steps):

        # Select an action using the current epsilon-greedy policy
        action = select_action(state, q, epsilon)

        # Interact with the environment
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated

        # Store the experience
        trajectory.append((state, action, reward, next_state, done))

        # Stop when the episode terminates
```

```
        if done:
            break

        # Continue from the next state
        state = next_state

    return np.array(trajectory, dtype=object)
```

We can now put everything together. The algorithm alternates two processes after each sampled episode:

sample episode ⟶ update Q ⟶ make behavior more greedy ⟶ sample again

We use first-visit MC, but now visits refer to state-action pairs rather than states.

```
def mc_control(env, gamma=0.99,
               init_alpha=0.5, min_alpha=0.01,
               init_epsilon=1.0, min_epsilon=0.1,
               decay_episodes=1000, n_episodes=3000,
               max_steps=200):

    # Learning-rate and exploration schedules
    alphas = decay_schedule(
        init_alpha, min_alpha, decay_episodes, n_episodes
    )

    epsilons = decay_schedule(
        init_epsilon, min_epsilon, decay_episodes, n_episodes
    )

    # Initialize the action-value function
    q = np.zeros(
        (env.observation_space, env.action_space),
        dtype=float
    )

    # Store the state-value estimates after each episode
    v_track = np.zeros(
        (n_episodes, env.observation_space),
        dtype=float
    )

    # Run one complete episode at a time
```

```
    for e in range(n_episodes):

        # Generate an episode using the current exploratory policy
        trajectory = generate_trajectory_control(
            select_action, q, env, epsilons[e], max_steps
        )

        # Compute all complete returns
        returns = compute_returns(trajectory[:, 2], gamma)

        # Track first visits to state-action pairs
        visited = np.zeros(
            (env.observation_space, env.action_space),
            dtype=bool
        )

        # Update each state-action pair at its first visit
        for t, (state, action, _, _, _) in enumerate(trajectory):

            if visited[state, action]:
                continue

            visited[state, action] = True

            # Monte Carlo target and error
            mc_target = returns[t]
            mc_error = mc_target - q[state, action]

            # Update the action-value estimate
            q[state, action] += alphas[e] * mc_error

        # Extract the greedy state-value estimate
        v = np.max(q, axis=1)

        # Store values for later analysis
        v_track[e] = v

    # Final greedy policy
    def pi(s):
        return np.argmax(q[s])

    return q, v, pi, v_track
```

The important difference from MC prediction is now explicit: prediction learns V for a fixed policy; control learns Q while simultaneously changing the policy used to collect experience. This is why exploration becomes essential. We can apply MC control to the Slippery Walk environment:

```
q_mc, v_mc, pi_mc, v_track_mc = mc_control(slippery_walk)

print('State   MC estimate   Optimal value')
print('-------------------------------------')
for s in range(1, 8):
    print(f'{s:^5}   {v_mc[s]:^11.3f}   {optimal_v[s]:^13.3f}')
```

```
State   MC estimate   Optimal value
-------------------------------------
  1        0.496          0.564
  2        0.726          0.763
  3        0.834          0.845
  4        0.886          0.889
  5        0.917          0.922
  6        0.944          0.952
  7        0.976          0.981
```

The optimal policy is straightforward for this environment, but the important point is that **the agent discovers it from interaction alone, without access to the transition probabilities**. We can also inspect how the value estimates evolve:

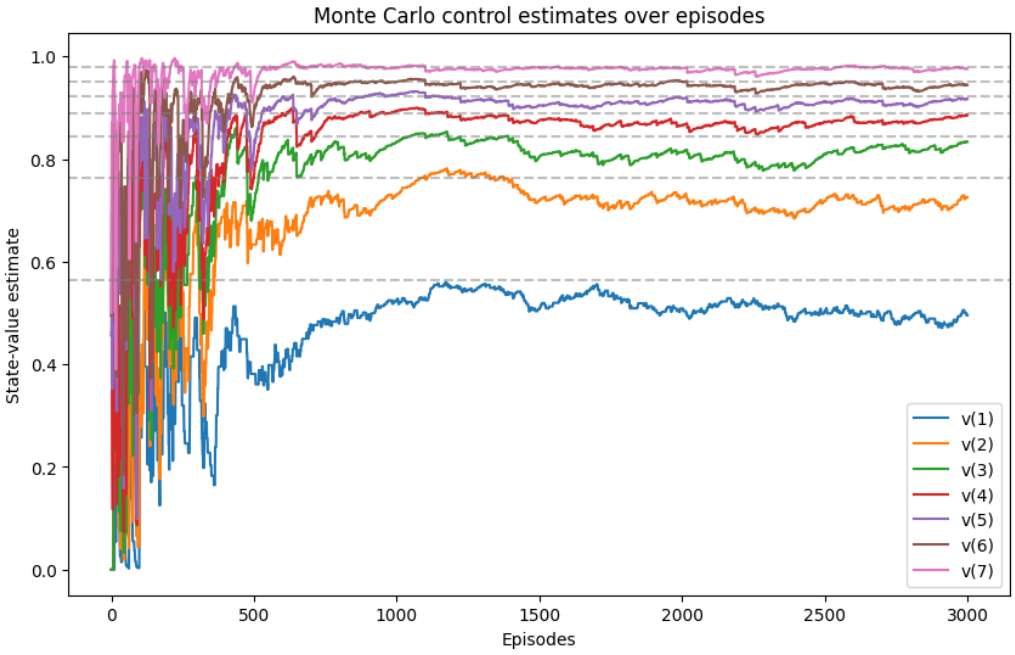
```
plt.figure(figsize=(10, 6))

# Learned estimates
plt.plot(v_track_mc[:, 1:8])

# Optimal values
plt.axhline(optimal_v[1], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[2], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[3], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[4], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[5], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[6], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[7], color='gray', linestyle='--', alpha=0.5)
```

```
plt.title('Monte Carlo control estimates over episodes')
plt.xlabel('Episodes')
plt.ylabel('State-value estimate')
plt.legend(['v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)', 'v(6)', 'v(7)'])

plt.show()
```



As in MC prediction, the estimates can be **quite noisy**. Each update uses a complete sampled return, so all the stochastic transitions encountered between a state-action pair and termination contribute to the target.

1.2.3 Improving policies after each step: SARSA

Monte Carlo control improves the policy **episode by episode**. It must wait until the episode terminates before the complete return is available and the action-value estimates can be updated. We already know how to remove this limitation from policy evaluation: r**eplace the complete Monte Carlo return with a one-step TD target**. We can apply exactly the same idea to action values, obtaining the **SARSA algorithm**. Instead of waiting for the complete return, SARSA observes one transition and then bootstraps from the estimated value of the next state-action pair:

$G_{t:t+1} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$

The action-value update becomes:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha_t \underbrace{\left[\underbrace{R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})}_{\text{SARSA target}} - Q(S_t, A_t) \right]}_{\text{SARSA error}}$$

Unlike MC control, the **update can be performed immediately after every transition**. SARSA can therefore be viewed as the TD counterpart of Monte Carlo control: both estimate Q and use epsilon-greedy exploration, but MC learns from complete returns whereas SARSA learns from one-step bootstrapped targets.The name SARSA comes directly from the five quantities involved in this update:

$S_t \rightarrow A_t \rightarrow R_{t+1} \rightarrow S_{t+1} \rightarrow A_{t+1}$

We can implement it as follows:

```
def sarsa(env, gamma=0.99,
          init_alpha=0.5, min_alpha=0.01,
          init_epsilon=1.0, min_epsilon=0.1,
          decay_episodes=1000, n_episodes=3000):

    # Calculate the learning-rate schedule
    alphas = decay_schedule(
        init_alpha, min_alpha, decay_episodes, n_episodes
    )

    # Calculate the exploration schedule
    epsilons = decay_schedule(
        init_epsilon, min_epsilon, decay_episodes, n_episodes
    )

    # Initialize the action-value function
    q = np.zeros(
        (env.observation_space, env.action_space),
        dtype=float
    )

    # Store the state-value estimates after each episode
    v_track = np.zeros(
        (n_episodes, env.observation_space),
        dtype=float
    )

    # Run one episode at a time
```

```
    for e in range(n_episodes):

        # Start a new episode
        state = env.reset()
        done = False

        # Select the first action using the current epsilon-greedy policy
        action = select_action(state, q, epsilons[e])

        while not done:

            # Execute the current action
            next_state, reward, terminated, truncated, info = env.step(action)
            done = terminated or truncated

            if done:
                # No bootstrapping from a terminal state
                sarsa_target = reward

            else:
                # Select the next action using the same epsilon-greedy policy
                next_action = select_action(
                    next_state, q, epsilons[e]
                )

                # One-step SARSA target
                sarsa_target = (
                    reward
                    + gamma * q[next_state, next_action]
                )

            # Compute the SARSA error
            sarsa_error = sarsa_target - q[state, action]

            # Update the current state-action value
            q[state, action] += alphas[e] * sarsa_error

            # Continue from the next state-action pair
            if not done:
                state, action = next_state, next_action

        # Extract the greedy state-value estimate
        v = np.max(q, axis=1)
```

```
        # Store values for later analysis
        v_track[e] = v

    # Extract the final greedy policy
    def pi(s):
        return np.argmax(q[s])

    return q, v, pi, v_track
```

Notice the fundamental difference in timing:

MC control: complete episode → update

SARSA: one transition → update → next transition → update → ...

We can now run SARSA on the Slippery Walk environment:

```
q_sarsa, v_sarsa, pi_sarsa, v_track_sarsa = sarsa(slippery_walk)

print('State   SARSA estimate   Optimal value')
print('---------------------------------------')
for s in range(1, 8):
    print(f'{s:^5}   {v_sarsa[s]:^14.3f}   {optimal_v[s]:^13.3f}')
```

```
State   SARSA estimate   Optimal value
---------------------------------------
  1         0.503            0.564
  2         0.740            0.763
  3         0.814            0.845
  4         0.869            0.889
  5         0.912            0.922
  6         0.943            0.952
  7         0.978            0.981
```

and inspect how its estimates evolve:

```
plt.figure(figsize=(10, 6))

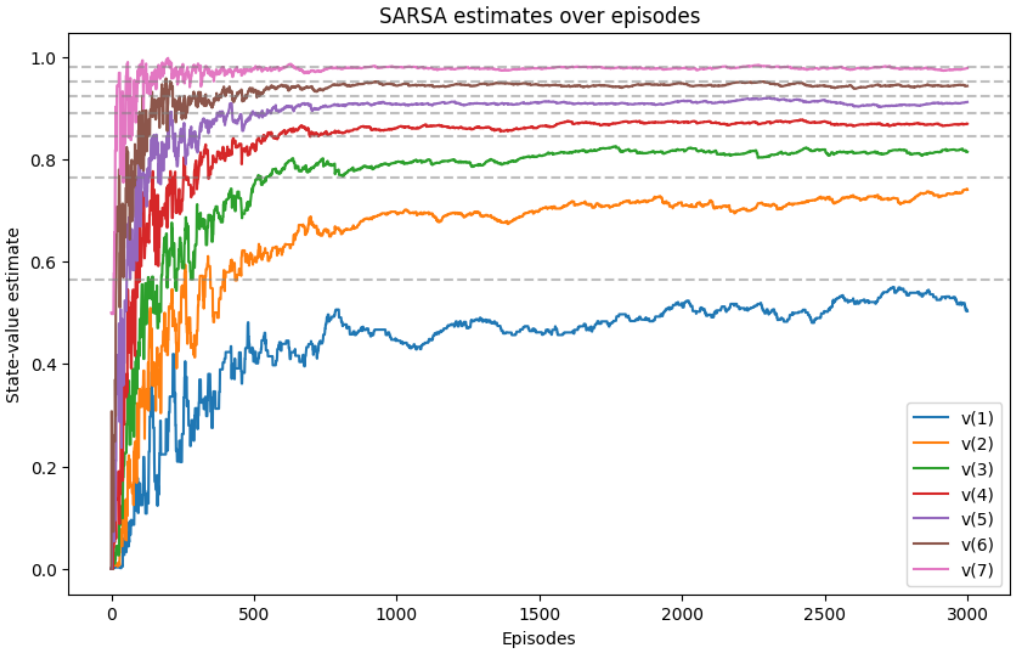
# Learned estimates
```

```
plt.plot(v_track_sarsa[:, 1:8])

# Optimal values
plt.axhline(optimal_v[1], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[2], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[3], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[4], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[5], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[6], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[7], color='gray', linestyle='--', alpha=0.5)

plt.title('SARSA estimates over episodes')
plt.xlabel('Episodes')
plt.ylabel('State-value estimate')
plt.legend(['v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)', 'v(6)', 'v(7)'])

plt.show()
```



SARSA is **substantially smoother** than Monte Carlo control. MC updates from complete sampled returns, accumulating the randomness of an entire trajectory, whereas SARSA bootstraps after a single transition and therefore uses **lower-variance targets**. The remaining gap and fluctuations are expected because exploration and learning continue throughout training.

1.2.4 Convergence requirements for on-policy control

Both Monte Carlo control and SARSA are **on-policy control methods**: they **learn action values from experience generated by the same policy they are trying to improve**. For these methods to converge to optimal behavior, two competing requirements must be satisfied: **the agent must keep exploring**, but **it must also eventually become greedy**. The first requirement is infinite exploration. Every state-action pair must continue to be visited:

$$\boxed{N_t(s,a) \to \infty \qquad \forall (s,a)}$$

Without this condition, the agent may stop trying an action before discovering its true value. An action that initially appears poor could therefore be permanently excluded from the policy. At the same time, exploration cannot remain equally strong forever if we want the behavior itself to become optimal. The policy must become greedy in the limit with respect to the learned action values:

$$\boxed{\pi_t(s) \to \arg\max_a ; Q_t(s,a)}$$

These two properties together are known as **GLIE (Greedy in the Limit with Infinite Exploration)**. The apparent tension is important: **exploration must progressively disappear, but slowly enough that every state-action pair is still visited infinitely often**. For an epsilon-greedy policy, this suggests a schedule in which:

$$\varepsilon_t \to 0$$

but not so quickly that exploration stops prematurely. A fixed positive ε, for example, guarantees continued exploration but does not become greedy in the limit. Conversely, setting ε to zero too early makes the policy greedy but may leave some actions insufficiently explored. On-policy MC control therefore uses exploration precisely because action values can only be learned for state-action pairs that are actually visited.

There is a second requirement concerning the learning rate. When we use incremental updates of the form

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha_k(S_t,A_t)\,[\text{target} - Q(S_t,A_t)]$$

the step sizes should progressively decrease while still allowing an unlimited amount of learning. The standard Robbins-Monro conditions, applied separately to each state-action pair, are:

$$\boxed{\sum_{k=1}^{\infty} \alpha_k(s,a) = \infty, \qquad \sum_{k=1}^{\infty} \alpha_k^2(s,a) < \infty}$$

The two conditions serve different purposes. The first,

$$\sum_k \alpha_k(s,a) = \infty$$

ensures that the total amount of possible future adjustment remains large enough to overcome arbitrary initial estimates. Learning must not stop too early.

The second,

$$\sum_k \alpha_k^2(s,a) < \infty$$

requires the updates to become sufficiently small so that the noise introduced by individual samples eventually averages out. A classical choice satisfying both conditions is:

$$\boxed{\alpha_k(s,a) = \frac{1}{N_k(s,a)}}$$

For Monte Carlo, this choice has an especially natural interpretation: it makes the incremental update exactly equivalent to computing the sample average of all returns observed for (s,a). Our experiments deliberately use a simpler finite-training setup. Both ε and α follow schedules indexed by episode, and both decay only to positive minimum values:

```
min_epsilon = 0.1
min_alpha   = 0.01
```

Therefore, our implementation does not strictly satisfy the asymptotic convergence conditions.

Because:

$$\varepsilon_{\min} > 0$$

the behavior policy always retains some exploration and never becomes completely greedy. Similarly, because:

$$\alpha_{\min} > 0$$

new samples continue to modify the estimates, so Q may continue to fluctuate slightly rather than settling exactly at a fixed point. This is a deliberate practical choice for our finite experiments: large values of ε and α encourage exploration and rapid learning initially, while smaller values later make behavior and estimates more stable. The theoretical conditions tell us what is required for asymptotic convergence. Our schedules are instead designed to obtain effective learning within a finite number of episodes.

1.2.5 Decoupling behavior from learning: Q-Learning

Learning does not have to be tied to the policy that generates the experience. We can separate two roles: a **behavior policy**, used to interact with the environment and generate experience,

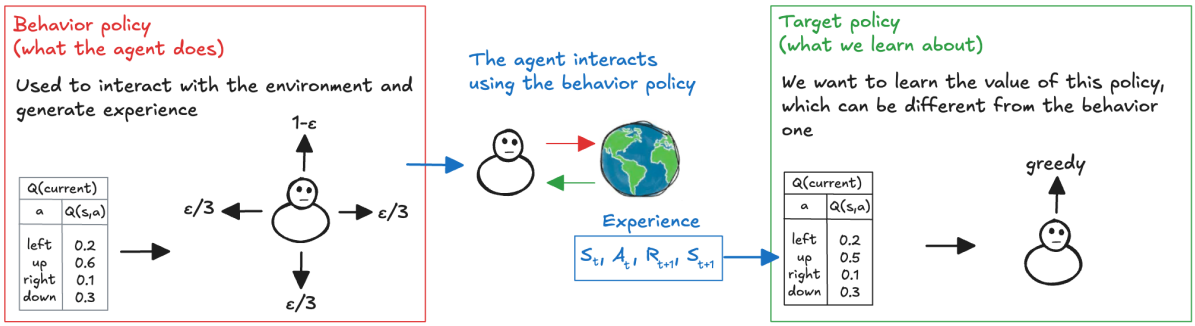
and a **target policy**, whose values we want to learn. The behavior policy determines **what the agent does**, while the target policy determines **what the agent learns about**. The experience is generated according to the behavior policy:

$$A_t \sim \pi_{\text{behaviour}}(S_t)$$

while the quantity we want to estimate is defined with respect to the target policy:

$$q_{\pi_{\text{target}}}(s,a) = \mathbb{E}_{\pi_{\text{target}}}[G_{t:T} \mid S_t = s, A_t = a]$$

This separation is powerful. For example, the behaviour policy can remain epsilon-greedy and continue exploring, while the target policy can be greedy. More generally, experience may come from an older version of the agent or from any other sufficiently exploratory policy.



But this creates a problem: **if experience is generated by a behaviour policy, how can we learn about a different target policy?** In general, off-policy methods need some mechanism to deal with this mismatch. **Q-Learning** provides a clever solution: instead of bootstrapping from the action that the behavior policy actually selects next, it **bootstraps directly from the action preferred by the greedy target policy**. SARSA uses the next action actually selected by the behavior policy:

$$G^{\text{SARSA}}_{t:t+1} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}), \qquad A_{t+1} \sim \pi_{\text{behaviour}}(S_{t+1})$$

Q-Learning instead uses **the action with the largest estimated value**:

$$\boxed{G^{\text{Q}}_{t:t+1} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a)}$$

which corresponds to the greedy target policy:

$$\boxed{\pi_{\text{target}}(s) = \arg\max_a Q(s,a)}$$

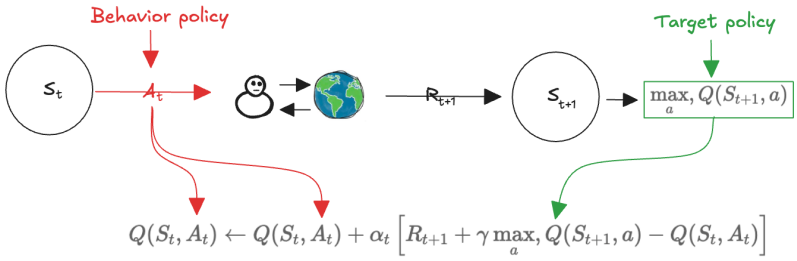
The transition:

$$S_t, A_t, R_{t+1}, S_{t+1}$$

is therefore generated by behaviour policy, while the target is constructed according to the particular target policy.

The Q-Learning update is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha_t \underbrace{\left[\underbrace{R_{t+1} + \gamma \max_a Q(S_{t+1}, a)}_{\text{Q-Learning target}} - Q(S_t, A_t) \right]}_{\text{Q-Learning error}}$$



In Q-Learning, we can therefore **behave exploratively while learning greedily**. We can implement it in Python:

```
def q_learning(env, gamma=0.99,
               init_alpha=0.5, min_alpha=0.01,
               init_epsilon=1.0, min_epsilon=0.1,
               decay_episodes=1000, n_episodes=3000):

    # Calculate the learning-rate schedule
    alphas = decay_schedule(
        init_alpha, min_alpha, decay_episodes, n_episodes
    )

    # Calculate the exploration schedule
    epsilons = decay_schedule(
        init_epsilon, min_epsilon, decay_episodes, n_episodes
    )

    # Initialize the action-value function
    q = np.zeros(
        (env.observation_space, env.action_space),
        dtype=float
    )

    # Store the state-value estimates after each episode
    v_track = np.zeros(
        (n_episodes, env.observation_space),
        dtype=float
```

```
    )

    # Run one episode at a time
    for e in range(n_episodes):

        # Start a new episode
        state = env.reset()
        done = False

        while not done:

            # Behavior policy: select an epsilon-greedy action
            action = select_action(state, q, epsilons[e])

            # Interact with the environment
            next_state, reward, terminated, truncated, info = env.step(action)
            done = terminated or truncated

            if done:
                # No bootstrapping from a terminal state
                q_learning_target = reward

            else:
                # Target policy: bootstrap from the greedy action
                q_learning_target = (
                    reward
                    + gamma * np.max(q[next_state])
                )

            # Compute the Q-Learning error
            q_learning_error = (
                q_learning_target - q[state, action]
            )

            # Update the action-value estimate
            q[state, action] += (
                alphas[e] * q_learning_error
            )

            # Continue from the next state
            state = next_state

        # Extract the greedy state-value estimate
```

```
        v = np.max(q, axis=1)

        # Store values for later analysis
        v_track[e] = v

    # Extract the final greedy policy
    def pi(s):
        return np.argmax(q[s])

    return q, v, pi, v_track
```

Notice that the behavior policy still explores according to ε. But ε never appears in the Q-Learning target. The target always uses the greedy action value:

$$\max_a Q(S_{t+1}, a)$$

This is the key distinction from SARSA. We can now run Q-Learning on the Slippery Walk environment:

```
q_ql, v_ql, pi_ql, v_track_ql = q_learning(slippery_walk)

print('State   QL estimate   Optimal value')
print('-------------------------------------')
for s in range(1, 8):
    print(f'{s:^5}   {v_ql[s]:^11.3f}   {optimal_v[s]:^13.3f}')
```

```
State   QL estimate   Optimal value
-------------------------------------
  1        0.549          0.564
  2        0.753          0.763
  3        0.844          0.845
  4        0.887          0.889
  5        0.923          0.922
  6        0.960          0.952
  7        0.985          0.981
```

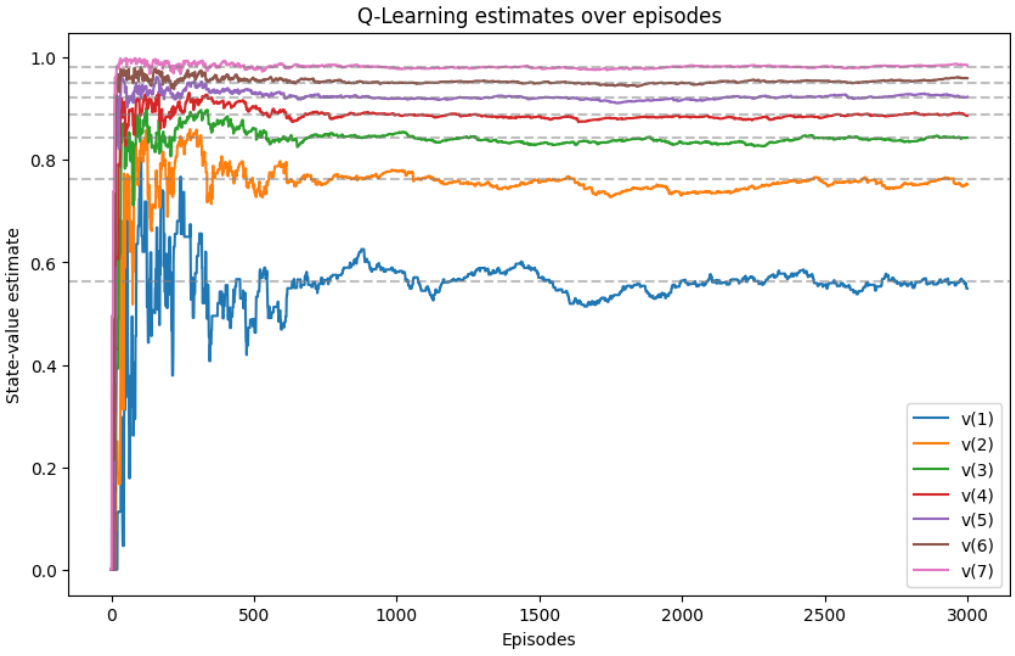
and inspect how its estimates evolve:

```
plt.figure(figsize=(10, 6))

# Learned estimates
plt.plot(v_track_ql[:, 1:8])

# Optimal values
plt.axhline(optimal_v[1], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[2], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[3], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[4], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[5], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[6], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[7], color='gray', linestyle='--', alpha=0.5)

plt.title('Q-Learning estimates over episodes')
plt.xlabel('Episodes')
plt.ylabel('State-value estimate')
plt.legend(['v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)', 'v(6)', 'v(7)'])
plt.show()
```



In this experiment, Q-Learning approaches the optimal values more rapidly than SARSA, particularly during the first few hundred episodes. This should not be interpreted as a general guarantee that off-policy methods learn faster. The difference comes from what the two algorithms learn while exploration is still active. SARSA is on-policy: its target includes the action

actually selected by the epsilon-greedy behavior policy. It therefore learns the value of the exploratory behavior itself. Q-Learning, instead, uses exploration only to generate experience, while its target always assumes greedy behavior from the next state onward. It therefore learns directly toward the value of the greedy target policy, even while the agent continues to explore. This distinction is visible in the plots: Q-Learning moves rapidly toward the optimal reference values, whereas SARSA approaches them more gradually. Both methods continue to exhibit some fluctuations because learning and exploration remain active throughout the experiment. It is also important to distinguish accurate value estimates from a good policy. The greedy policy depends only on which action has the largest estimated value:

$\pi(s) = \arg\max_a Q(s,a)$

Therefore, the Q estimates do not need to be numerically exact to produce the optimal policy. As long as the correct action has the largest estimated value in each state, the learned greedy policy can already be optimal even while the value estimates are still changing.

1.2.6 Convergence requirements for off-policy control

Off-policy learning changes the relationship between exploration and optimality. In on-policy methods, the same policy must satisfy two apparently conflicting requirements: it must keep exploring, but it must also become greedy in the limit. This is why we need GLIE. In Q-Learning, t**hese two responsibilities are decoupled**. The behavior policy is responsible for generating sufficiently rich experience, while the target policy is responsible for defining the behavior we want to learn: **The behavior policy therefore does not need to become greedy**. It only needs to ensure that every state-action pair continues to receive enough experience:

$N_t(s,a) \to \infty \qquad \forall (s,a)$

For example, it may remain epsilon-greedy: exploration can continue indefinitely without changing what Q-Learning is trying to learn. **Greediness is instead built directly into the target policy** through the Q-Learning target:

$\pi_{\text{target}}(s) = \arg\max_a Q(s,a)$

$G^Q_{t:t+1} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$

Thus, unlike on-policy control, GLIE is not required for the behavior policy. The behavior policy can remain exploratory because the target used for learning is already greedy. The second requirement remains the same. For each state-action pair, the learning rates must satisfy the Robbins-Monro conditions:

$$\sum_{k=1}^{\infty} \alpha_k(s,a) = \infty$$
$$\sum_{k=1}^{\infty} \alpha_k^2(s,a) < \infty$$

As before, the first condition ensures that learning never stops too early, while the second ensures that the influence of sampling noise eventually vanishes. Under the usual assumptions, sufficient state-action visitation together with appropriate diminishing step sizes guarantees convergence of Q-Learning to the optimal action-value function.

1.2.7 Solving the maximization bias: Double Q-Learning

Q-Learning has an important weakness: it can **systematically overestimate action values**. Recall its target:

$G^Q_{t:t+1} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$

The problem lies in the max operation. What we really want is the **maximum true action value**, but we do not know the true values. Instead, we take **the maximum over their noisy estimates**:

$\max_a Q(S_{t+1}, a)$

These are not the same thing. In general:

$\mathbb{E}\left[\max_a Q(s,a)\right] \geq \max_a \mathbb{E}[Q(s,a)]$

Why does this create an upward bias? Suppose all actions actually have value zero:

$q(s,a_1) = q(s,a_2) = \cdots = q(s,a_n) = 0$

but our estimates are noisy:

$Q(s,a_1) = 0.11, \quad Q(s,a_2) = 0.65, \quad Q(s,a_3) = -0.44, \quad Q(s,a_4) = -0.26, \ldots$

The true maximum is:

$\max_a q(s,a) = 0$

but Q-Learning uses:

$\max_a Q(s,a) = 0.65$

The problem is not simply that the estimates contain errors. Some estimates are too high and others too low. The problem is that **maximization systematically selects the positive errors**. The same estimates are being used both to answer: Which action looks best? and How valuable is that action? This produces the **maximization bias** of Q-Learning. **Double Q-Learning** addresses this problem by maintaining **two separate estimates**:

$Q_1(s,a)$
$Q_2(s,a)$

The central idea is simple: **use one estimate to select the action and the other to evaluate it**. Suppose we decide to update Q1. First, Q1 selects the action it considers best:

$$A^* = \arg\max_a Q_1(S_{t+1}, a)$$

but Q1 is not allowed to evaluate its own choice. Instead, we ask Q2 for the value of that action:

$$Q_2(S_{t+1}, A^*)$$

The target therefore becomes:

$$\boxed{G^{DQ_1}_{t:t+1} = R_{t+1} + \gamma Q_2\left(S_{t+1}, \arg\max_a Q_1(S_{t+1}, a)\right)}$$

and we update Q1:

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha\left[G^{DQ_1}_{t:t+1} - Q_1(S_t, A_t)\right]$$

On another transition, we reverse the roles. Q2 selects:

$$A^* = \arg\max_a Q_2(S_{t+1}, a)$$

while Q1 evaluates:

$$\boxed{G^{DQ_2}_{t:t+1} = R_{t+1} + \gamma Q_1\left(S_{t+1}, \arg\max_a Q_2(S_{t+1}, a)\right)}$$

The two estimators are therefore used symmetrically. At each transition, we randomly choose which one to update. The intuition is that an action may be selected because Q1 happens to overestimate it, but there is no reason for the independent estimation error in Q2 to overestimate exactly the same action. By separating selection from evaluation, Double Q-Learning **breaks the mechanism responsible for systematic positive maximization bias**.



We can implement this idea in Python:

```
def double_q_learning(env, gamma=0.99,
                      init_alpha=0.5, min_alpha=0.01,
                      init_epsilon=1.0, min_epsilon=0.1,
```

```
                    decay_episodes=1000, n_episodes=3000):

    # Calculate the learning-rate schedule
    alphas = decay_schedule(
        init_alpha, min_alpha, decay_episodes, n_episodes
    )

    # Calculate the exploration schedule
    epsilons = decay_schedule(
        init_epsilon, min_epsilon, decay_episodes, n_episodes
    )

    # Initialize two independent action-value estimates
    q1 = np.zeros(
        (env.observation_space, env.action_space),
        dtype=float
    )

    q2 = np.zeros(
        (env.observation_space, env.action_space),
        dtype=float
    )

    # Store state-value estimates for later analysis
    v_track = np.zeros(
        (n_episodes, env.observation_space),
        dtype=float
    )

    # Run one episode at a time
    for e in range(n_episodes):

        state = env.reset()
        done = False

        while not done:

            # Use both estimates to define the behavior policy
            q = (q1 + q2) / 2
            action = select_action(state, q, epsilons[e])

            # Interact with the environment
            next_state, reward, terminated, truncated, info = env.step(action)
```

```
            done = terminated or truncated

            # Randomly choose which Q-function to update
            if np.random.randint(2):

                if done:
                    dq_target = reward

                else:
                    # Q1 selects the action
                    best_action = np.argmax(q1[next_state])

                    # Q2 evaluates the action
                    dq_target = (
                        reward
                        + gamma * q2[next_state, best_action]
                    )

                # Update Q1
                dq_error = dq_target - q1[state, action]
                q1[state, action] += alphas[e] * dq_error

            else:

                if done:
                    dq_target = reward

                else:
                    # Q2 selects the action
                    best_action = np.argmax(q2[next_state])

                    # Q1 evaluates the action
                    dq_target = (
                        reward
                        + gamma * q1[next_state, best_action]
                    )

                # Update Q2
                dq_error = dq_target - q2[state, action]
                q2[state, action] += alphas[e] * dq_error

            state = next_state
```

```
        # Combine the two estimates
        q = (q1 + q2) / 2
        v = np.max(q, axis=1)

        # Store values for later analysis
        v_track[e] = v

    # Extract the final greedy policy
    def pi(s):
        return np.argmax(q[s])

    return q, v, pi, v_track
```

We can now run Double Q-Learning on the Slippery Walk environment:

```
q_dql, v_dql, pi_dql, v_track_dql = double_q_learning(slippery_walk)

print('State   Double Q estimate   Optimal value')
print('----------------------------------------')

for s in range(1, 8):
    print(f'{s:^5}   {v_dql[s]:^17.3f}   {optimal_v[s]:^13.3f}')
```

```
State   Double Q estimate   Optimal value
----------------------------------------
  1           0.556             0.564
  2           0.753             0.763
  3           0.848             0.845
  4           0.893             0.889
  5           0.927             0.922
  6           0.953             0.952
  7           0.982             0.981
```

and inspect the estimates:

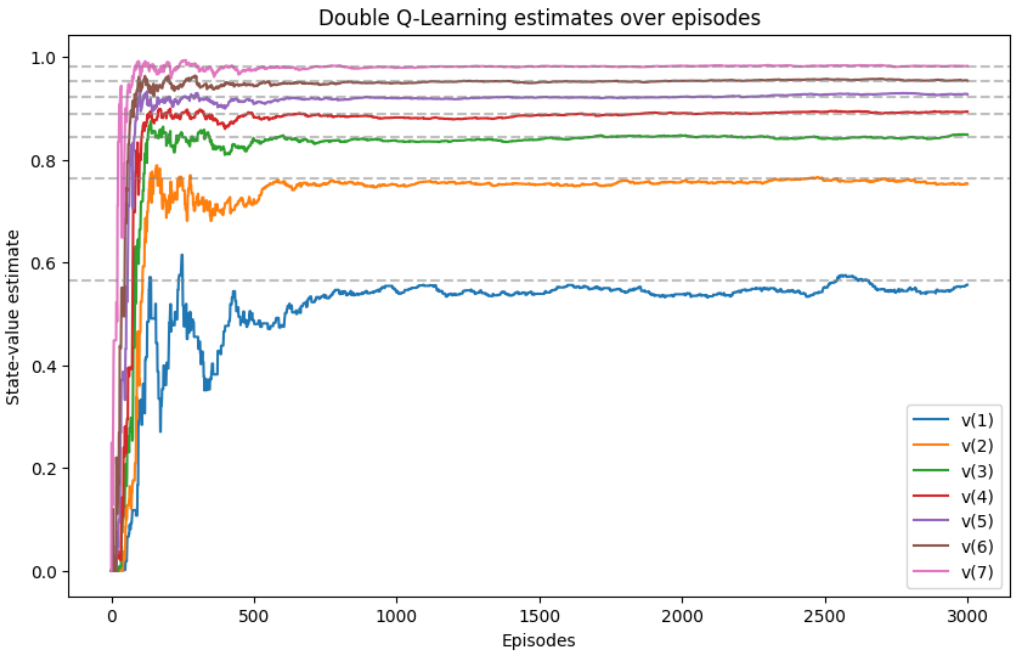
```
plt.figure(figsize=(10, 6))

plt.plot(v_track_dql[:, 1:8])
```

```
# Optimal values
plt.axhline(optimal_v[1], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[2], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[3], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[4], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[5], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[6], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[7], color='gray', linestyle='--', alpha=0.5)

plt.title('Double Q-Learning estimates over episodes')
plt.xlabel('Episodes')
plt.ylabel('State-value estimate')
plt.legend(['v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)', 'v(6)', 'v(7)'])

plt.show()
```



In this experiment, Double Q-Learning may approach the optimal values somewhat differently from ordinary Q-Learning, but the important effect is that the systematic upward pressure produced by the max operator is reduced. The resulting estimates can therefore be **more reliable when action values are noisy**. Double Q-Learning requires two action-value tables instead of one, **doubling the storage requirement**.

1.2.8 Using multi-step estimates: SARSA(lambda)

SARSA learns from a one-step target. After observing a transition, it updates only the current state-action pair:

$G_{t:t+1}^{\text{SARSA}} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$

But the same idea that led from TD(0) to TD(λ) in evaluation can also be applied to control. Instead of assigning the current TD error only to the most recent state-action pair, we can **propagate it backward to previously visited state-action pairs**. This gives us **SARSA**(λ). The one-step SARSA error remains:

$\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$

but we now associate an eligibility trace with every state-action pair:

$E(s, a)$

Instead of the eligibility vector used by TD(λ) to track states, SARSA(λ) therefore uses an **eligibility matrix** with the same shape as the Q-table. When a pair state-action is visited, its trace is increased, making that pair eligible to receive credit from subsequent TD errors. The error is then propagated to all eligible state-action pairs:

$\boxed{Q \leftarrow Q + \alpha, \delta_t E}$

Finally, the traces decay:

$\boxed{E \leftarrow \gamma \lambda E}$

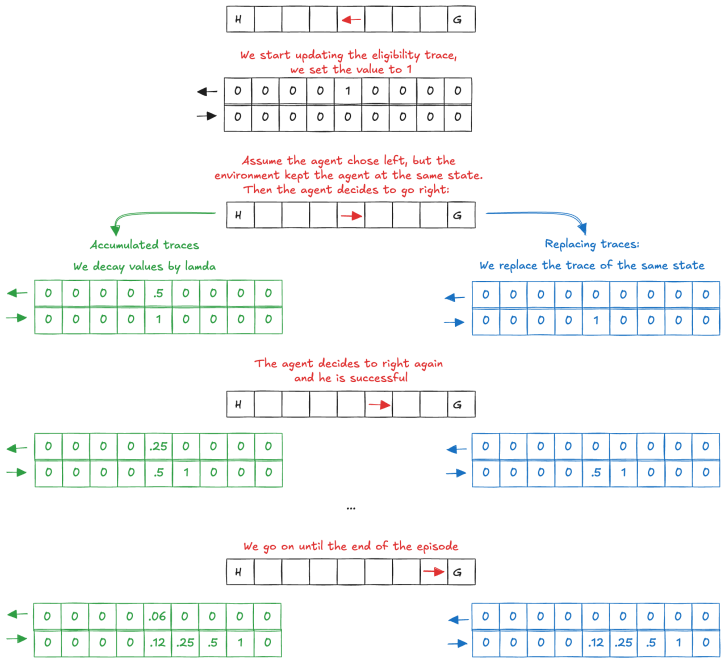
Thus, a reward observed now can modify not only the most recent decision, but also decisions made several steps earlier. The parameter λ controls **how far backward credit propagates**: small values emphasize recent decisions, whereas larger values preserve eligibility for longer. There is an additional choice when a state-action pair is visited more than once before its previous eligibility has disappeared. With **accumulating traces**, every visit adds one:

$\boxed{E(S_t, A_t) \leftarrow E(S_t, A_t) + 1}$

A repeatedly visited pair can therefore have an eligibility greater than one. Its trace captures both **recency and frequency**: a pair receives more credit if it was visited recently, but also if it was visited repeatedly. With **replacing traces**, a new visit replaces the previous eligibility:

$\boxed{E(S_t, A_t) \leftarrow 1}$

so repeated visits do not make the trace grow beyond one.



The question is therefore intuitive: if the same decision occurs repeatedly, should it become more eligible because it occurred many times, or should it simply remain fully eligible because it occurred recently? Accumulating traces emphasize both frequency and recency, whereas replacing traces emphasize primarily recency and prevent eligibility from growing excessively when the agent repeatedly visits the same state-action pair. This can be particularly useful when trajectories contain loops. For SARSA, there is also a stronger version of replacing traces in which, when action is selected in a state, the traces associated with the other actions in the same state are cleared. This is the variant implemented below:

$$E(S_t, a) \leftarrow 0 \quad \forall a \neq A_t, \qquad E(S_t, A_t) \leftarrow 1$$

We can implement SARSA(λ) as follows:

```
def sarsa_lambda(env, gamma=0.99,
                 init_alpha=0.5, min_alpha=0.01,
                 init_epsilon=1.0, min_epsilon=0.1,
                 lambda_=0.5, replacing_traces=True,
                 decay_episodes=1000, n_episodes=3000):

    # Calculate the learning-rate schedule
    alphas = decay_schedule(
        init_alpha, min_alpha, decay_episodes, n_episodes
    )
```

```
    # Calculate the exploration schedule
    epsilons = decay_schedule(
        init_epsilon, min_epsilon, decay_episodes, n_episodes
    )

    # Initialize the action-value function
    q = np.zeros(
        (env.observation_space, env.action_space),
        dtype=float
    )

    # Initialize one eligibility trace for every state-action pair
    E = np.zeros(
        (env.observation_space, env.action_space),
        dtype=float
    )

    # Store state-value estimates for later analysis
    v_track = np.zeros(
        (n_episodes, env.observation_space),
        dtype=float
    )

    # Run one episode at a time
    for e in range(n_episodes):

        # Reset all eligibility traces at the beginning of the episode
        E.fill(0)

        # Start a new episode
        state = env.reset()
        done = False

        # Select the initial action
        action = select_action(state, q, epsilons[e])

        while not done:

            # Execute the current action
            next_state, reward, terminated, truncated, info = env.step(action)
            done = terminated or truncated

            if done:
```

```
            # No bootstrapping from a terminal state
            sarsa_lambda_target = reward

        else:
            # Select the next action using the same behavior policy
            next_action = select_action(
                next_state, q, epsilons[e]
            )

            # Compute the one-step SARSA target
            sarsa_lambda_target = (
                reward
                + gamma * q[next_state, next_action]
            )

        # Compute the one-step SARSA error
        sarsa_lambda_error = (
            sarsa_lambda_target - q[state, action]
        )

        # Update the eligibility trace
        if replacing_traces:

            # Clear traces for the other actions in this state
            E[state].fill(0)

            # Make the current state-action pair fully eligible
            E[state, action] = 1

        else:

            # Accumulate eligibility
            E[state, action] += 1

        # Propagate the TD error to all eligible state-action pairs
        q += alphas[e] * sarsa_lambda_error * E

        # Decay all eligibility traces
        E *= gamma * lambda_

        # Continue from the next state-action pair
        if not done:
            state, action = next_state, next_action
```

```
        # Extract the greedy state-value estimate
        v = np.max(q, axis=1)

        # Store values for later analysis
        v_track[e] = v

    # Extract the final greedy policy
    def pi(s):
        return np.argmax(q[s])

    return q, v, pi, v_track
```

We can apply SARSA(λ) to the Slippery Walk environment:

```
q_sarsa_lambda, v_sarsa_lambda, pi_sarsa_lambda, v_track_sarsa_lambda = sarsa_lambda(
    slippery_walk
)

print('State   SARSA(lambda)   Optimal value')
print('---------------------------------------')

for s in range(1, 8):
    print(f'{s:^5}   {v_sarsa_lambda[s]:^13.3f}   {optimal_v[s]:^13.3f}')
```

```
State   SARSA(lambda)   Optimal value
---------------------------------------
  1         0.544           0.564
  2         0.734           0.763
  3         0.823           0.845
  4         0.874           0.889
  5         0.912           0.922
  6         0.946           0.952
  7         0.981           0.981
```

and inspect how its estimates evolve:

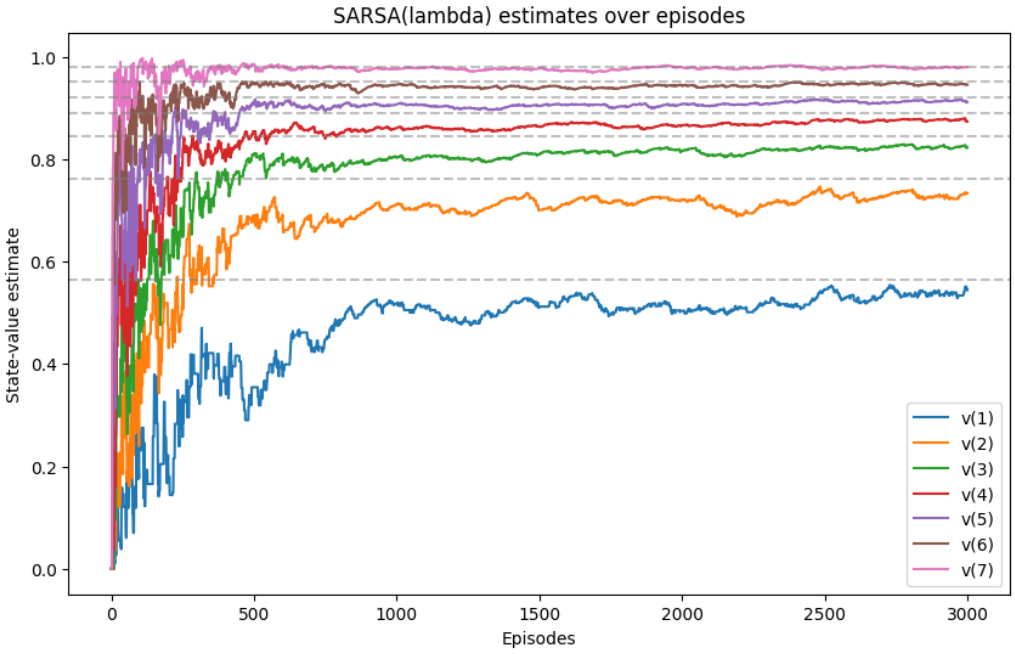
```
plt.figure(figsize=(10, 6))
```

```
plt.plot(v_track_sarsa_lambda[:, 1:8])

# Optimal values
plt.axhline(optimal_v[1], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[2], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[3], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[4], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[5], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[6], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[7], color='gray', linestyle='--', alpha=0.5)

plt.title('SARSA(lambda) estimates over episodes')
plt.xlabel('Episodes')
plt.ylabel('State-value estimate')
plt.legend(['v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)', 'v(6)', 'v(7)'])

plt.show()
```



The advantage of SARSA(λ) is **faster temporal credit assignment**. Ordinary SARSA propagates new information backward essentially one transition at a time. With eligibility traces, a single TD error can immediately update many state-action pairs that contributed to reaching the current situation. This can be **particularly useful when rewards are delayed**: a reward received near the end of a trajectory can immediately affect decisions made several steps earlier instead of requiring repeated episodes for that information to propagate backward. The parameter λ con-

trols the temporal extent of this credit assignment. When $\lambda = 0$, all previous traces disappear and the algorithm reduces to ordinary SARSA. As λ increases, TD errors propagate further into the past. The trade-off is **additional memory and computation**: SARSA(λ) must maintain and update an eligibility trace for every state-action pair, and λ introduces another hyperparameter whose value can substantially affect learning.

1.2.9 Using multi-step estimates and decoupling behaviour from learning: Q(lambda)

We can combine the two ideas introduced so far: **multi-step credit assignment through eligibility traces** and **off-policy learning**. This leads to **Watkins's Q**(λ), an extension of Q-Learning that uses eligibility traces to propagate TD errors backward to previously visited state-action pairs. As in ordinary Q-Learning, the one-step target remains greedy:

$$G^{Q}_{t:t+1} = R_{t+1} + \gamma \max_a, Q(S_{t+1}, a)$$

and the TD error is:

$$\delta_t = R_{t+1} + \gamma \max_a, Q(S_{t+1}, a) - Q(S_t, A_t)$$

As in SARSA(λ), we maintain an eligibility trace for every state-action pair and use the current TD error to update all eligible pairs:

$$\boxed{Q \leftarrow Q + \alpha, \delta_t E}$$

The difficulty is that Q-Learning is off-policy. The behavior policy may be exploratory, while the target policy is greedy:

$$A_{t+1} \sim \pi_{\text{behaviour}}(S_{t+1})$$
$$\pi_{\text{target}}(S_{t+1}) = \arg\max_a, Q(S_{t+1}, a)$$

As long as the behavior policy selects a greedy action, the observed trajectory is still compatible with the greedy target policy. We can therefore preserve the previous traces and decay them normally:

$$E \leftarrow \gamma\lambda E \qquad \text{if } A_{t+1} \text{ is greedy}$$

But suppose exploration causes the behavior policy to select a non-greedy action. From that point onward, the sampled trajectory no longer follows the greedy target policy we are trying to learn. Watkins's solution is simple, **cut the traces**:

$$E \leftarrow 0 \qquad \text{if } A_{t+1} \text{ is non-greedy}$$

The intuition is important. Eligibility traces propagate information backward through a sequence of decisions. In an off-policy method, we want this multi-step propagation to represent the target policy, not the exploratory behavior policy. Therefore, **the trace can extend backward**

only while the sampled actions remain consistent with the greedy target policy. As soon as exploration takes us off that path, the multi-step connection is broken. Notice that this does not mean that the exploratory transition is discarded. The current state-action pair still receives the ordinary Q-Learning update. What is interrupted is the propagation of future TD errors through the previous eligibility traces. We can implement the algorithm as follows:

```
def q_lambda(env, gamma=0.99,
             init_alpha=0.5, min_alpha=0.01,
             init_epsilon=1.0, min_epsilon=0.1,
             lambda_=0.5, replacing_traces=True,
             decay_episodes=1000, n_episodes=3000):

    # Calculate the learning-rate schedule
    alphas = decay_schedule(
        init_alpha, min_alpha, decay_episodes, n_episodes
    )

    # Calculate the exploration schedule
    epsilons = decay_schedule(
        init_epsilon, min_epsilon, decay_episodes, n_episodes
    )

    # Initialize the action-value function
    q = np.zeros(
        (env.observation_space, env.action_space),
        dtype=float
    )

    # Initialize eligibility traces
    E = np.zeros(
        (env.observation_space, env.action_space),
        dtype=float
    )

    # Store state-value estimates for later analysis
    v_track = np.zeros(
        (n_episodes, env.observation_space),
        dtype=float
    )

    # Run one episode at a time
    for e in range(n_episodes):
```

```
        # Reset eligibility traces at the beginning of each episode
        E.fill(0)

        # Start a new episode
        state = env.reset()
        done = False

        # Preselect the action because we will need to determine
        # whether the next selected action is greedy
        action = select_action(state, q, epsilons[e])

        while not done:

            # Execute the current action
            next_state, reward, terminated, truncated, info = env.step(action)
            done = terminated or truncated

            if done:

                # No bootstrapping from a terminal state
                q_lambda_target = reward
                next_action_is_greedy = False

            else:

                # Select the next action using the behavior policy
                next_action = select_action(
                    next_state, q, epsilons[e]
                )

                # Check whether the behavior action is also greedy
                next_action_is_greedy = (
                    q[next_state, next_action]
                    == np.max(q[next_state])
                )

                # Q-Learning still uses the greedy target
                q_lambda_target = (
                    reward
                    + gamma * np.max(q[next_state])
                )

            # Compute the Q-Learning TD error
```

```
            q_lambda_error = (
                q_lambda_target - q[state, action]
            )

            # Update the eligibility of the current pair
            if replacing_traces:

                # Clear traces for other actions in the same state
                E[state].fill(0)

                # Make the current pair fully eligible
                E[state, action] = 1

            else:

                # Accumulating trace
                E[state, action] += 1

            # Apply the TD error to all eligible pairs
            q += alphas[e] * q_lambda_error * E

            # Continue traces only if behavior remains greedy
            if next_action_is_greedy:
                E *= gamma * lambda_
            else:
                E.fill(0)

            # Continue from the next state-action pair
            if not done:
                state, action = next_state, next_action

        # Extract the greedy state-value estimate
        v = np.max(q, axis=1)

        # Store values for later analysis
        v_track[e] = v

    # Extract the final greedy policy
    def pi(s):
        return np.argmax(q[s])

    return q, v, pi, v_track
```

We can apply Watkins's Q(λ) to the Slippery Walk environment:

```
q_ql_lambda, v_ql_lambda, pi_ql_lambda, v_track_ql_lambda = q_lambda(slippery_walk)

print('State   Q(lambda) estimate   Optimal value')
print('-----------------------------------------')
for s in range(1, 8):
    print(f'{s:^5}   {v_ql_lambda[s]:^18.3f}   {optimal_v[s]:^13.3f}')
```

```
State   Q(lambda) estimate   Optimal value
-----------------------------------------
  1           0.568              0.564
  2           0.763              0.763
  3           0.841              0.845
  4           0.888              0.889
  5           0.921              0.922
  6           0.953              0.952
  7           0.982              0.981
```

and visualize the estimates:

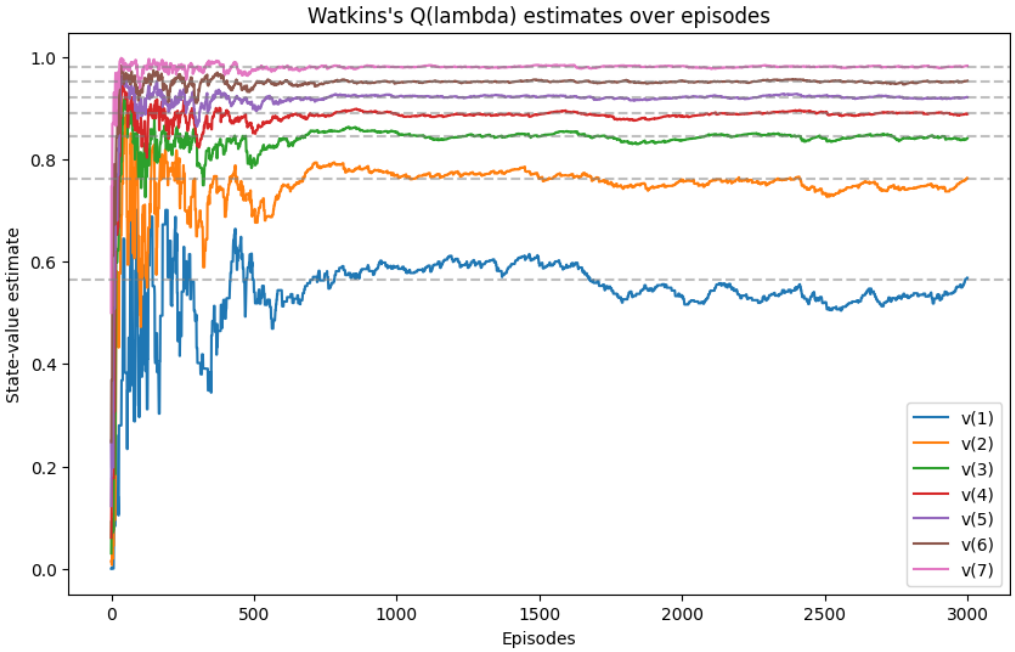
```
plt.figure(figsize=(10, 6))

plt.plot(v_track_ql_lambda[:, 1:8])

# Optimal values
plt.axhline(optimal_v[1], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[2], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[3], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[4], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[5], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[6], color='gray', linestyle='--', alpha=0.5)
plt.axhline(optimal_v[7], color='gray', linestyle='--', alpha=0.5)

plt.title("Watkins's Q(lambda) estimates over episodes")
plt.xlabel('Episodes')
plt.ylabel('State-value estimate')
plt.legend(['v(1)', 'v(2)', 'v(3)', 'v(4)', 'v(5)', 'v(6)', 'v(7)'])

plt.show()
```



Watkins's Q(λ) can propagate new information backward through several previous state-action pairs, potentially improving credit assignment when rewards are delayed. Eligibility-trace methods can therefore **learn more efficiently per unit of experience** in problems with long temporal dependencies. There is, however, an important interaction between exploration and trace length. Every non-greedy exploratory action cuts the trace. If the behavior policy explores frequently, long traces may therefore be interrupted often, reducing the advantage of a large λ. As exploration decreases, longer sequences of greedy actions become possible and the algorithm can exploit longer traces more effectively. Thus, Watkins's Q(λ) combines the two central ideas developed in this section: Q-Learning decouples behavior from the greedy target policy, while eligibility traces propagate credit across multiple steps—but only as long as the sampled behavior remains consistent with that target policy.

1.3 Comparison

We can now compare the algorithms by measuring how closely their learned state values approach the optimal value function. For each episode, we compute the Mean Absolute Error (MAE) over the seven non-terminal states:

$$\text{MAE}(V, v_*) = \frac{1}{7} \sum_{s=1}^{7} |V(s) - v_*(s)|$$

A lower MAE means that the learned value function is closer to the optimal one. Because individual learning curves can be noisy, we apply a moving average to make their overall trends

easier to compare.

```
def moving_average(a, n=100):

    ret = np.cumsum(a, dtype=float)
    ret[n:] = ret[n:] - ret[:-n]

    return ret[n - 1:] / n
```

We first compute the MAE using only the non-terminal states:

```
mae_mc = np.mean(np.abs(v_track_mc[:, 1:8] - optimal_v[1:8]), axis=1)
mae_sarsa = np.mean(np.abs(v_track_sarsa[:, 1:8] - optimal_v[1:8]), axis=1)
mae_ql = np.mean(np.abs(v_track_ql[:, 1:8] - optimal_v[1:8]), axis=1)
mae_dql = np.mean(np.abs(v_track_dql[:, 1:8] - optimal_v[1:8]), axis=1)
mae_sarsa_lambda = np.mean(np.abs(v_track_sarsa_lambda[:, 1:8] - optimal_v[1:8]),
 ↪  axis=1)
mae_ql_lambda = np.mean(np.abs(v_track_ql_lambda[:, 1:8] - optimal_v[1:8]), axis=1)
```

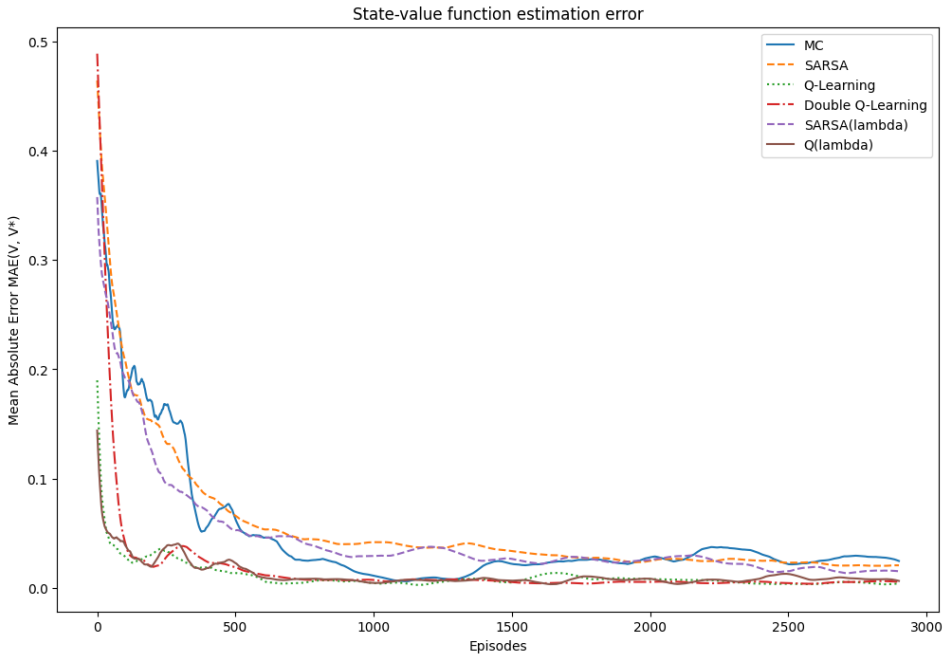
and compare the smoothed learning curves:

```
plt.figure(figsize=(12, 8))

plt.plot(moving_average(mae_mc), label='MC')
plt.plot(moving_average(mae_sarsa), linestyle='--', label='SARSA')
plt.plot(moving_average(mae_ql), linestyle=':', label='Q-Learning')
plt.plot(moving_average(mae_dql), linestyle='-.', label='Double Q-Learning')
plt.plot(moving_average(mae_sarsa_lambda), linestyle='--', label='SARSA(lambda)')
plt.plot(moving_average(mae_ql_lambda), label='Q(lambda)')

plt.title('State-value function estimation error')
plt.xlabel('Episodes')
plt.ylabel('Mean Absolute Error MAE(V, V*)')
plt.legend()

plt.show()
```



The results show a clear difference in how quickly the algorithms reduce the value-estimation error. All methods eventually reach relatively small errors, but they follow **substantially different learning trajectories**. The two methods using eligibility traces improve particularly quickly during the early stages of training. By propagating each TD error backward to several recently visited state-action pairs, they can distribute new information across the trajectory immediately instead of waiting for it to propagate one step at a time. Q-Learning and Double Q-Learning also reduce the error very rapidly. Double Q-Learning shows a sharp initial decrease and reaches a very small error within the first few hundred episodes. Q-Learning also approaches the optimal values quickly and eventually maintains one of the lowest errors. Their off-policy targets allow them to learn toward greedy behavior even while the behavior policy continues to explore. SARSA improves more gradually. Because it is on-policy, its estimates reflect the exploratory behavior actually being followed during training. As exploration decreases, its estimates progressively approach those of the greedy policy. Monte Carlo is the slowest and most irregular method in this experiment, which is consistent with its reliance on complete sampled returns rather than bootstrapped estimates. After roughly the first thousand episodes, however, the differences become much smaller: all methods remain within a relatively narrow error range. The residual error and fluctuations are expected because our learning-rate and exploration schedules stop at positive minimum values rather than vanishing completely. The most important conclusion is therefore not that one algorithm is universally superior, but that the different mechanisms have a visible effect on learning dynamics. One-step bootstrapping propagates information faster than complete-return MC in this problem; eligibility traces can accelerate this propagation further; and off-policy methods can learn toward greedy values while exploration

is still taking place. These curves should still be interpreted as the result of this particular run, environment, and set of hyperparameters. Since Slippery Walk and the learning algorithms are stochastic, a stronger comparison would repeat the complete experiment over many independent runs and plot the average MAE, ideally together with a measure of variability.